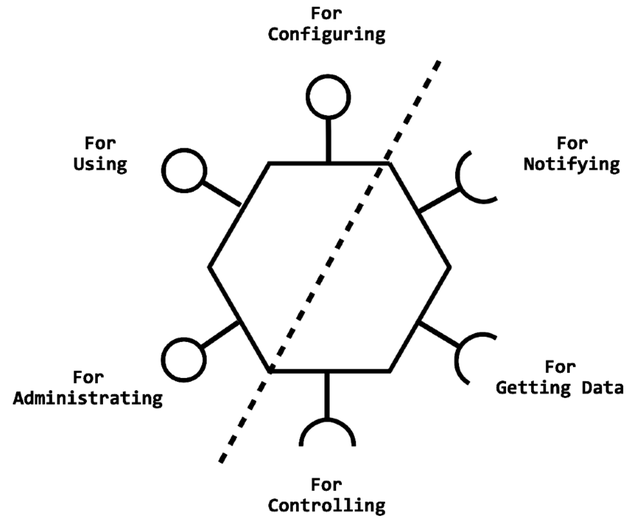


Hexagonal Architecture

Ports and Adapter Patterns Apply

Olarn U.
ODT

https://github.com/olarn/Hexagonal-public

Topics

Subjects to be learned in this course

- 9:00 - 10:30 - Introduction
  - Application Architecture
  - Separation of Concerns: Dependencies Injection
  - GoF OOP Design patterns Revisit
  - Hexagonal Architecture - AKA Ports & Adapters Pattern
  - Workshop: Refactoring by Apply Ports & Adapters
- 10:45 - 12:00 - Lab 1: Refactoring by Apply Ports & Adapters
  - Understand incremental
    from Test-to-Test to Real-to-Real
  - Workshop - Apply ports and adapters
- 13:00 - 14:15 - From Big Ball of Mud to Modular Monolith
  - Layer Architecture vs. Modular Architecture
  - Workshop :- From Big Ball of Mud to Modular Monolith with Ports & Adapters Pattern
- 14:30 - 15:30 - Distributed Architecture
  - Monolith vs. Distributed Architecture
  - Workshop: Split module into a new Service
- 15:30 - 16:15 - Wrap up
  - Hexagonal Architecture Wrap up
  - Hexagonal vs. Clean Architecture
  - DDD Related
  - Q&A
- 16:15 - 16:30 - Class Retrospective

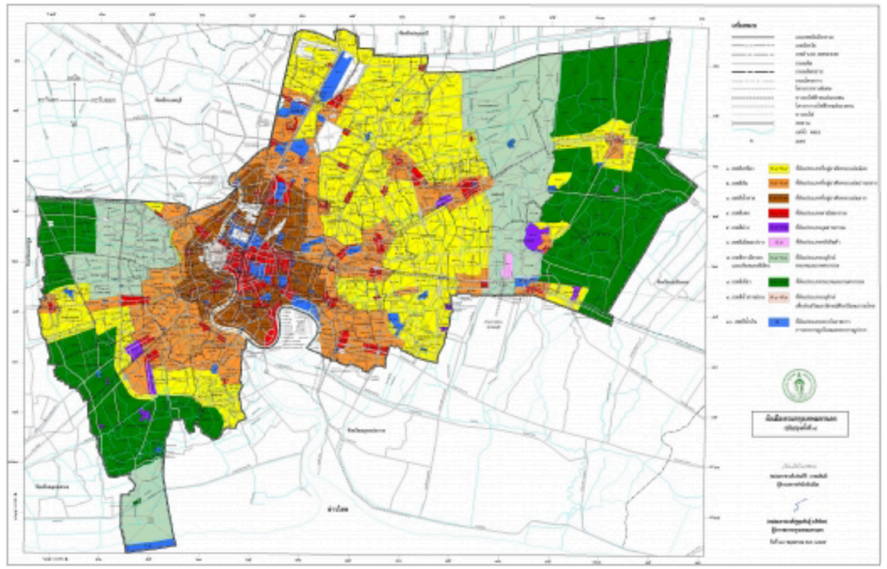
Hexagonal Architecture
aka.
Ports & Adapters Pattern

Application Architecture

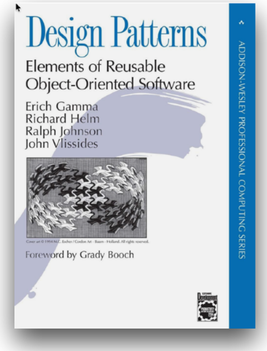
Shaping the structure of the application software

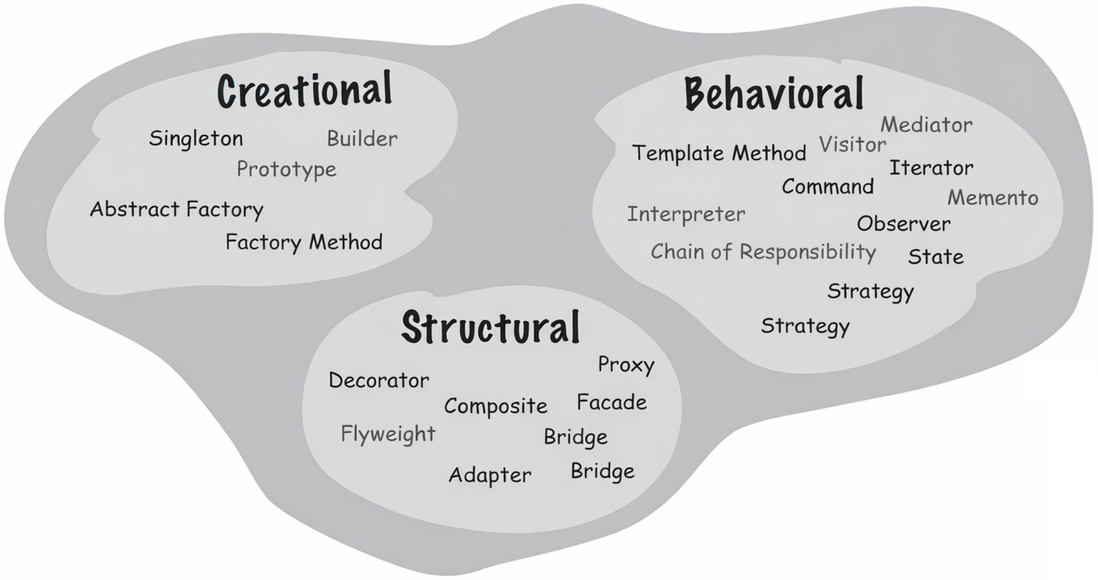
- Software today is more than just code— it’s an ecosystem of modules, databases, APIs, and external systems.
- Without a clear structure, projects often end up as a “Big Ball of Mud”, where every change becomes risky and slow.
- Architecture provides principles and boundaries that help teams balance flexibility, performance, and maintainability.
- What Application Architecture means
  - It’s the logical organizing of business logic, data, and interfaces within a single application boundary.
  - It defines how components communicate, how we isolate changes, and how we express business rules clearly.
  - Good architecture enables us to answer the question:
    - “Where should this logic go?”
    - “How do we replace technology without breaking the business?”

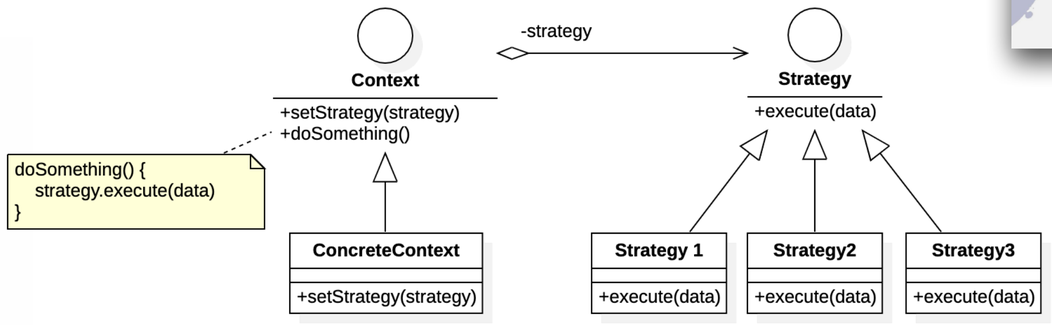
Amsterdam, Netherland



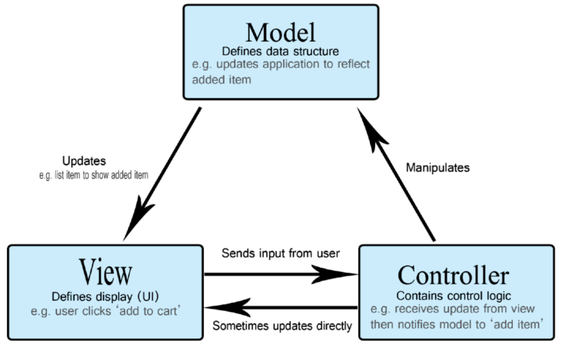
Application & Design Patterns

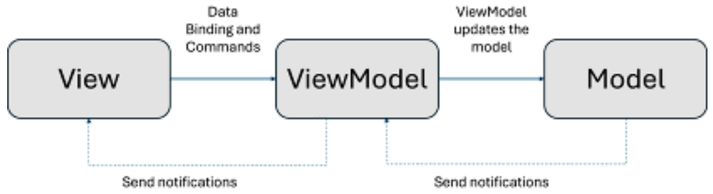






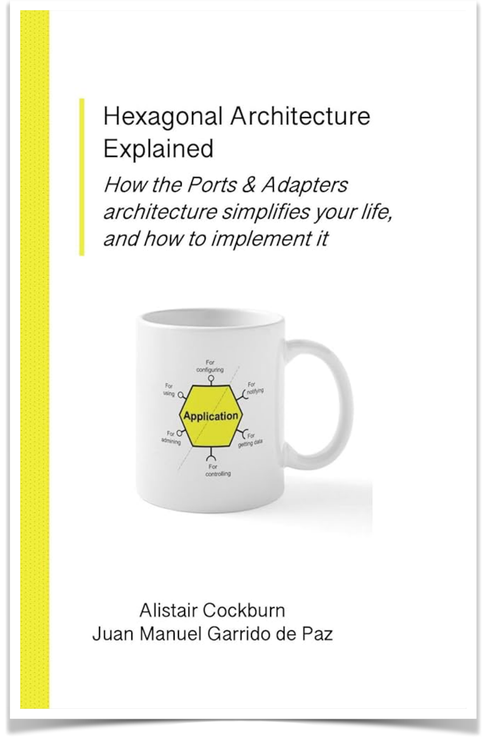
Strategy Pattern





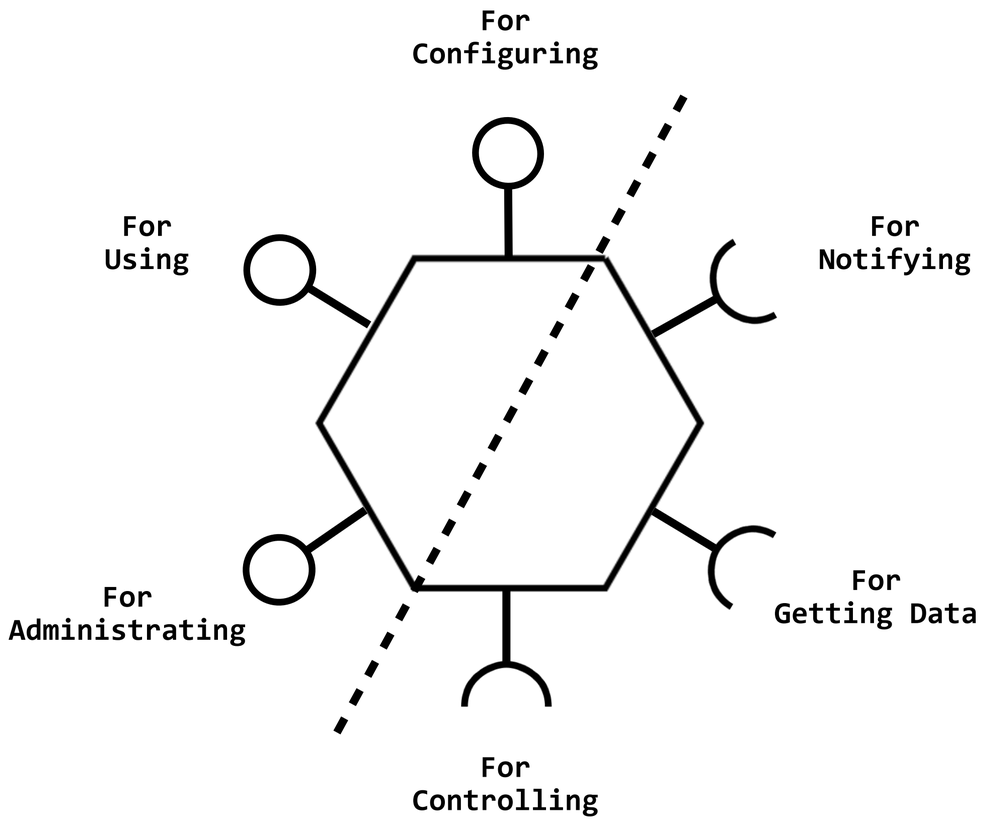
Hexagonal Architecture

by Alistair Cockburn



Ports & Adapters Pattern

Overview



- Ports & Adapters is an architectural pattern that **separates** the core business logic of an application from the external systems it interacts with — such as databases, user interfaces, and APIs — by introducing abstract interfaces called ***ports*** and concrete implementations called ***adapters***.
- **Ports** define what the application can do (the interfaces of the core).
- **Adapters** define how the application connects to the outside world.
- This allows the core logic to remain independent of frameworks, databases, or UI technology.

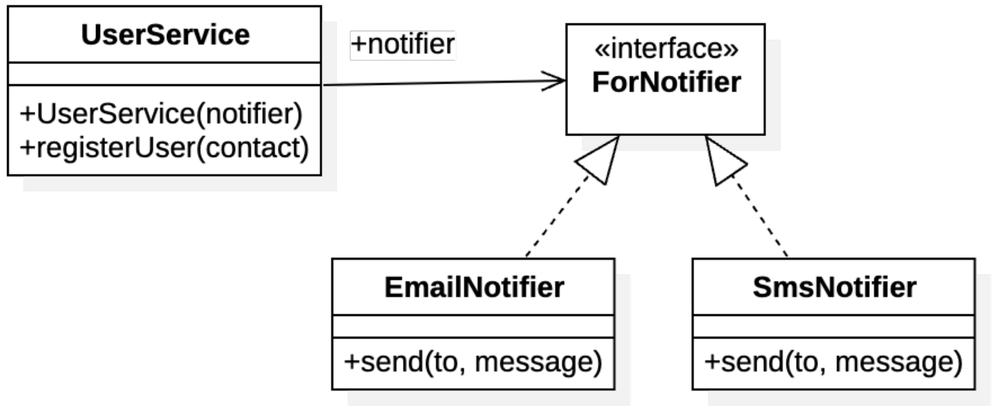
Dependency Injection

1/2 - Solve dependency problem for this class…


```
class EmailSender {
  send(to: string, message: string) {
    console.log(`Email sent to ${to} with message: ${message}`)
  }
}

class UserService {
  registerUser(email: string) {
    const emailSender = new EmailSender()
    emailSender.send(email, 'Welcome to our service!')
  }
}
```

Dependency Injection

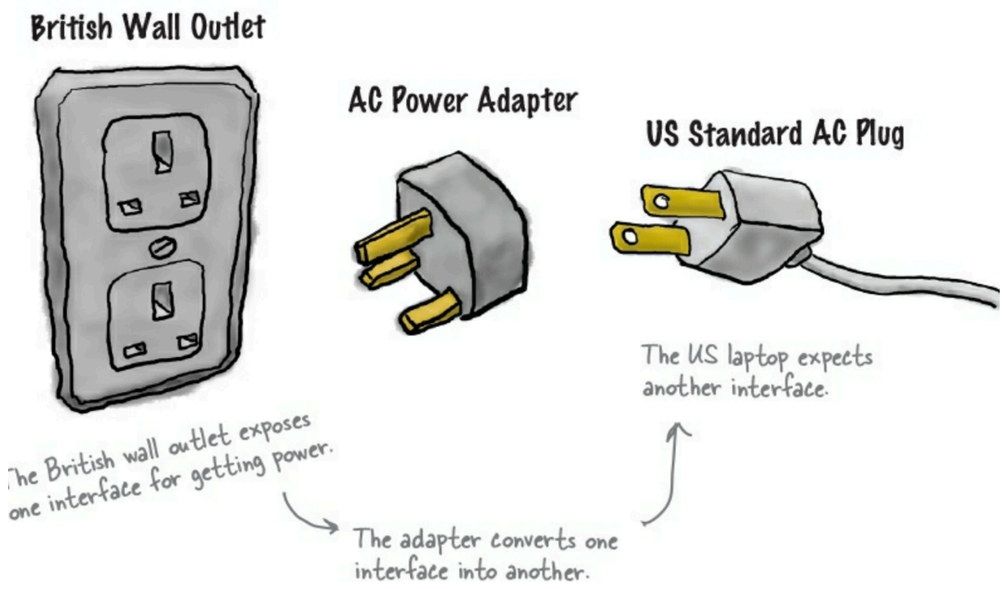
2/2 - Solve dependency problem for this class…



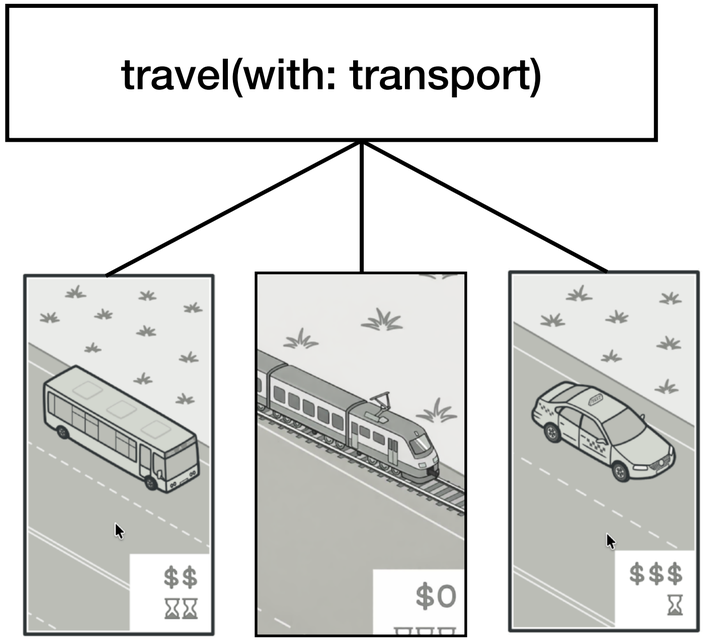

```
interface ForNotifier {
  send(to: string, message: string): void
}

class UserService2 {
  constructor(private readonly notifier: ForNotifier) {}

  registerUser(email: string) {
    this.notifier.send(email, 'Welcome to our service!')
  }
}

class EmailNotifier implements ForNotifier {
  send(to: string, message: string) {
    console.log(`Email sent to ${to} with message: ${message}`)
  }
}

class SmsNotifier implements ForNotifier {
  send(to: string, message: string) {
    console.log(`SMS sent to ${to} with message: ${message}`)
  }
}
```

Adapter & Strategy Pattern

OOP Design Pattern - GoF



Adapter Pattern



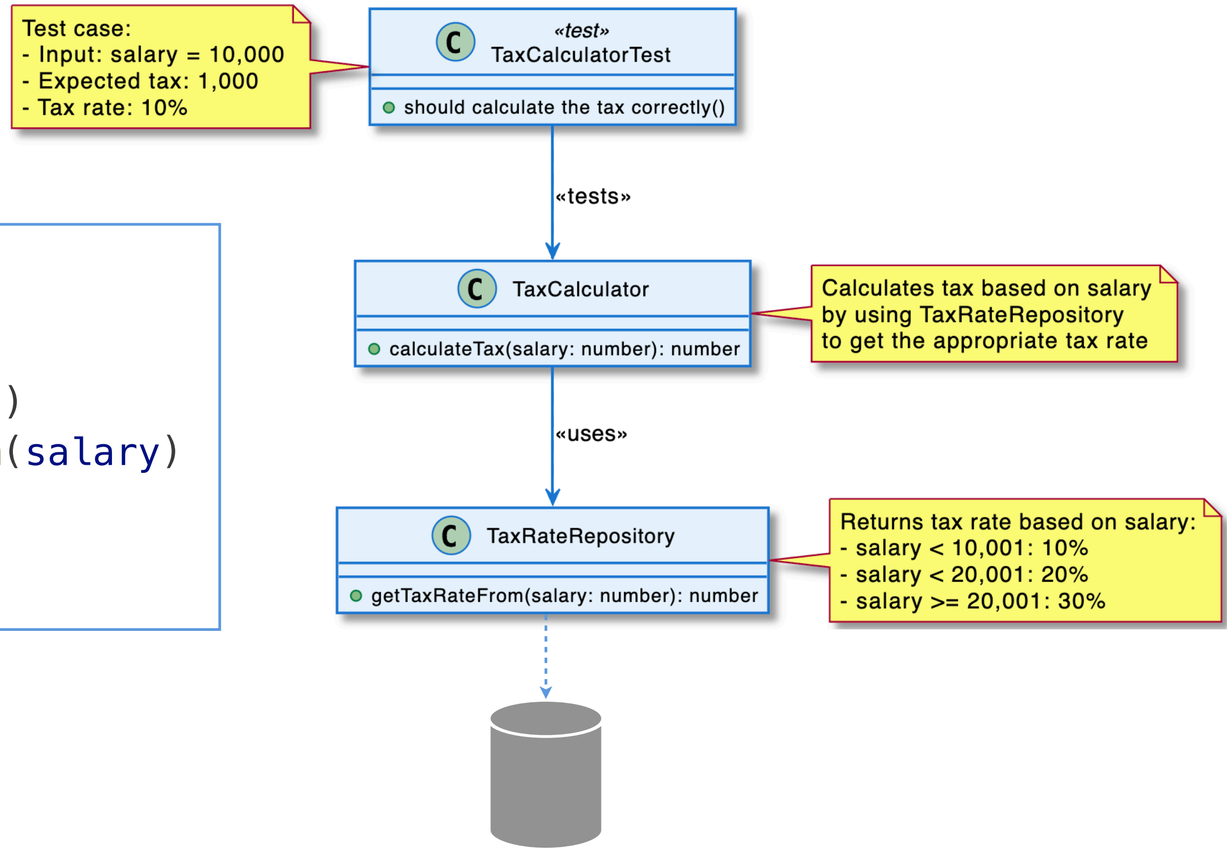
Strategy Pattern

Refactoring with Ports & Adaptors Pattern

Lab: Tax Calculator

Overview

```
export class TaxCalculator {
  calculateTax(salary: number): number {
    const taxRepository = new TaxRateRepository()
    const taxRate = taxRepository.getTaxRateFrom(salary)
    return salary * taxRate
  }
}
```



Discussion: What could be problem about the TaxCalculator class?

Lab: Tax Calculator

1/3 - Existing code

```ts
// taxCalculator.ts

export class TaxCalculator {
  calculateTax(salary: number): number {
    const taxRepository = new TaxRateRepository()
    const taxRate = taxRepository.getTaxRateFrom(salary)
    return salary * taxRate
  }
}

export class TaxRateRepository {
  getTaxRateFrom(salary: number): number {
    if (salary < 10001) {
      return 0.1
    } else if (salary < 20001) {
      return 0.2
    } else {
      return 0.3
    }
  }
}
```

```ts
// taxCalculator.test.ts

import { TaxCalculator } from "./taxCalculator"

describe('TaxCalculator', () => {
  it('should calculate the tax correctly', () => {
    const taxCalculator = new TaxCalculator()
    const tax = taxCalculator.calculateTax(10000)
    expect(tax).toBe(1000)
  })
})
```

Lab: Tax Calculator

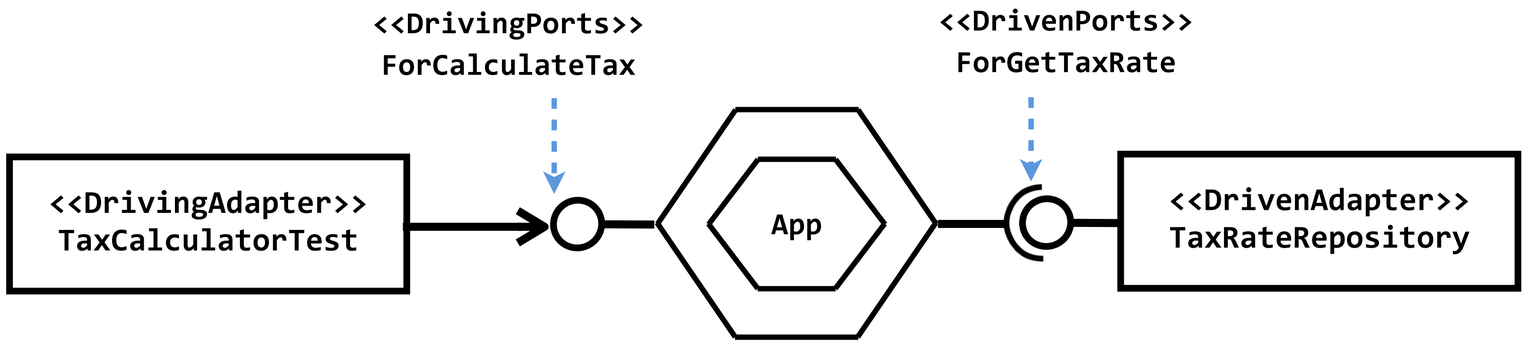
2/3 - Instruction

1. Move **`TaxCalculator`** into folder **`TaxCalculatorApp`**.
2. Add interface class - **`ForCalculateTax`** - in the folder **`DrivingPorts`.**
3. Implement **`TaxCalculator`** class to implement **`ForCalculateTax`** interface.
4. Update unit test to use **`ForCalculateTax`** type.

5. Add interface class - **`ForGetTaxRate`** - in the folder **`DrivenPorts`**.
6. Implement ***`TaxRateRepository`*** class to implement **`ForGetTaxRate`** interface.
7. Implement dependency injection for the **`TaxCalculator`** class to accept instance of the **`ForGetTaxRate`** interface.
8. Update unit test to inject mock object of the **`ForGetTaxRate`** into the constructor.

Lab: Tax Calculator

3/3 - Solution - Ports & Adapters Pattern




```ts
// Driving Port

export interface ForCalculateTax {
  calculateTax(salary: number): number
}


// Driven Port

export interface ForGetTaxRate {
  getTaxRateFrom(salary: number): number
}
```

```ts
// App

import { ForGetTaxRate } from "../DrivenPorts/forGetTaxRate"
import { ForCalculateTax } from "../DrivingPorts/forCalculateTax"

export class TaxCalculator implements ForCalculateTax {
  constructor(private readonly forGetTaxRate: ForGetTaxRate) { }

  calculateTax(salary: number): number {
    const taxRate = this.forGetTaxRate.getTaxRateFrom(salary)
    return salary * taxRate
  }
}
```

Apply Ports & Adapters

The Development Sequence

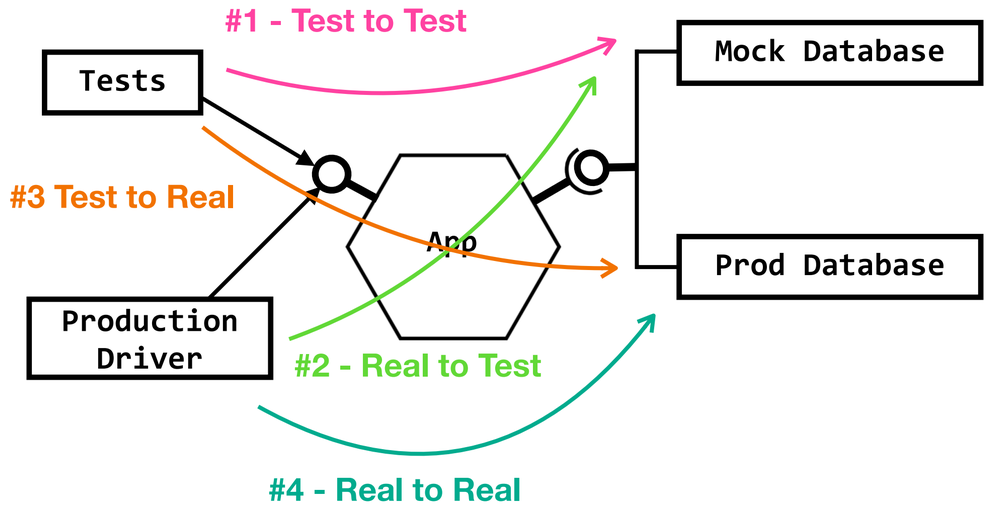
The brief description of the development sequence is as follows:

1. Test-to-test: Create the test as the first driver of the app, and connect it to a test double (in memory, mock, stub, fake, etc).

2. Real-to-test: Put the production driver in place, connected to the test double.

3. Test-to-real: Connect tests to the production database, or a test database that uses the same technology as the production database.

4. Real-to-real: Finally, when everything else works, connect the production drive and the production database.



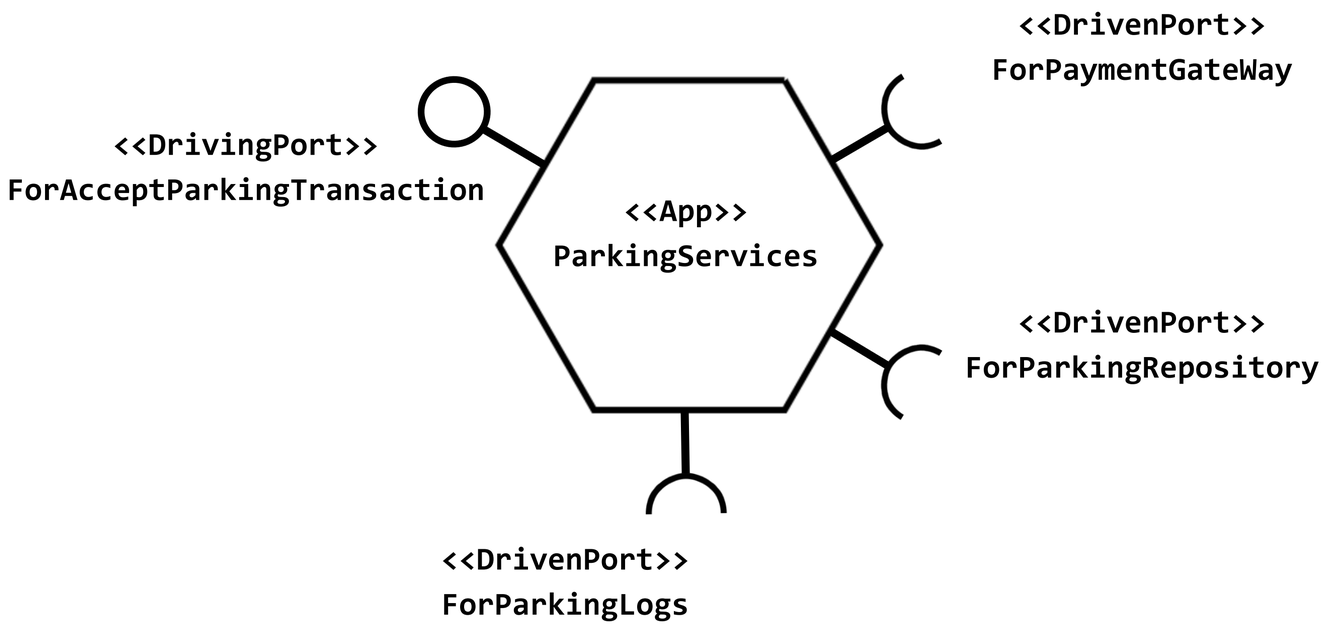
Exercise: Parking Services

Requirement

- Implement ParkingServices to accept ParkingTransaction.
  - Save to database.
  - Call Payment Gateway.
  - Write log for transaction consolidate with PaymentGateway.
- Apply Ports & Adapters that needed.

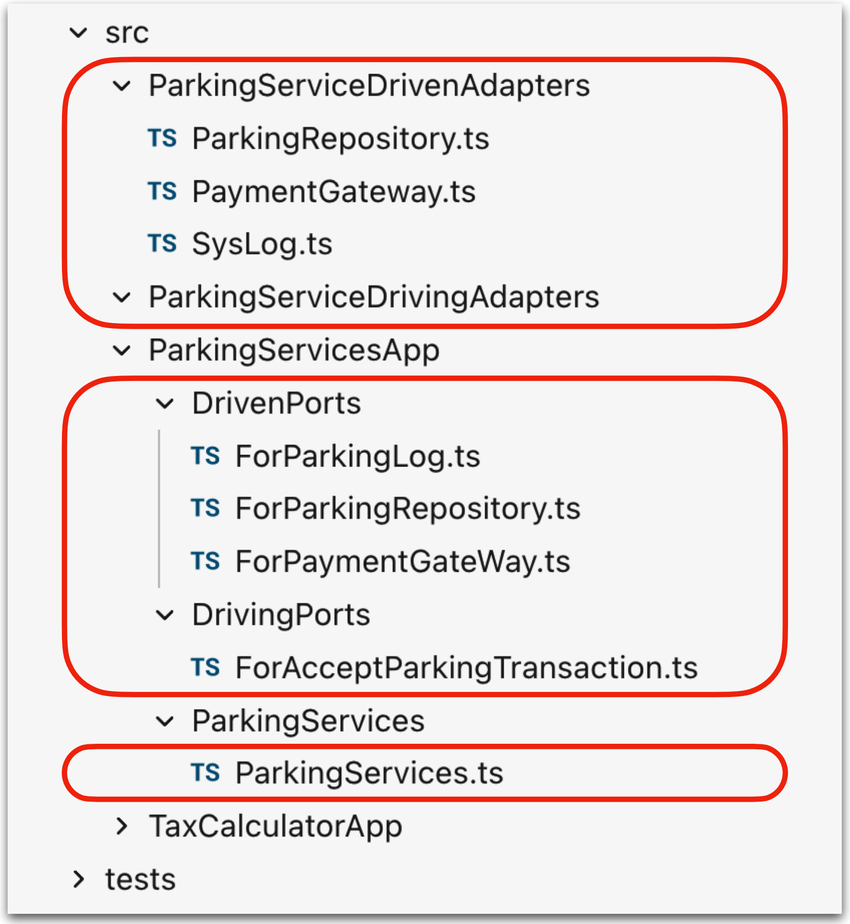
```
type ParkingTransaction = Readonly<{
    plateNo: string
    parkingZone: string
    paymentPayload: PaymentPayload
}>
```

```
type PaymentPayload = Readonly<{
    transaction: string
    cardNo: string
    expireDate: Date
    ccv: string
}>
```



Decoupling

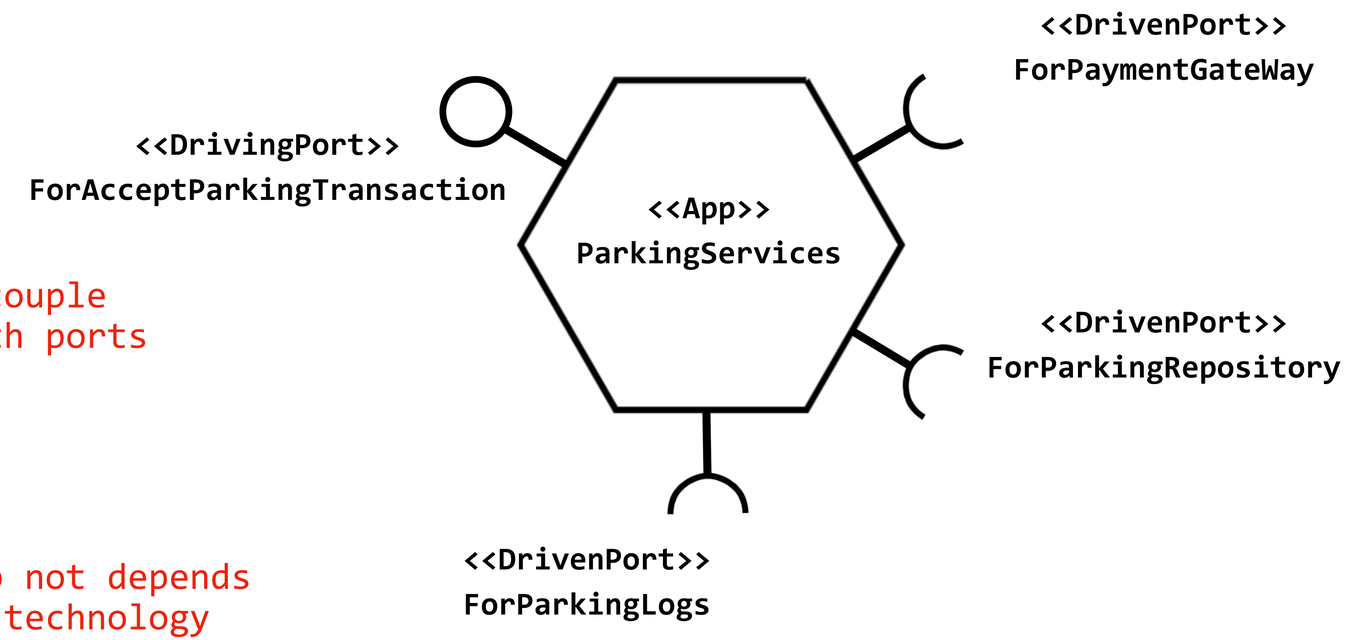
App, Driver, and Driving are not depends on each other



decoupled
adapters

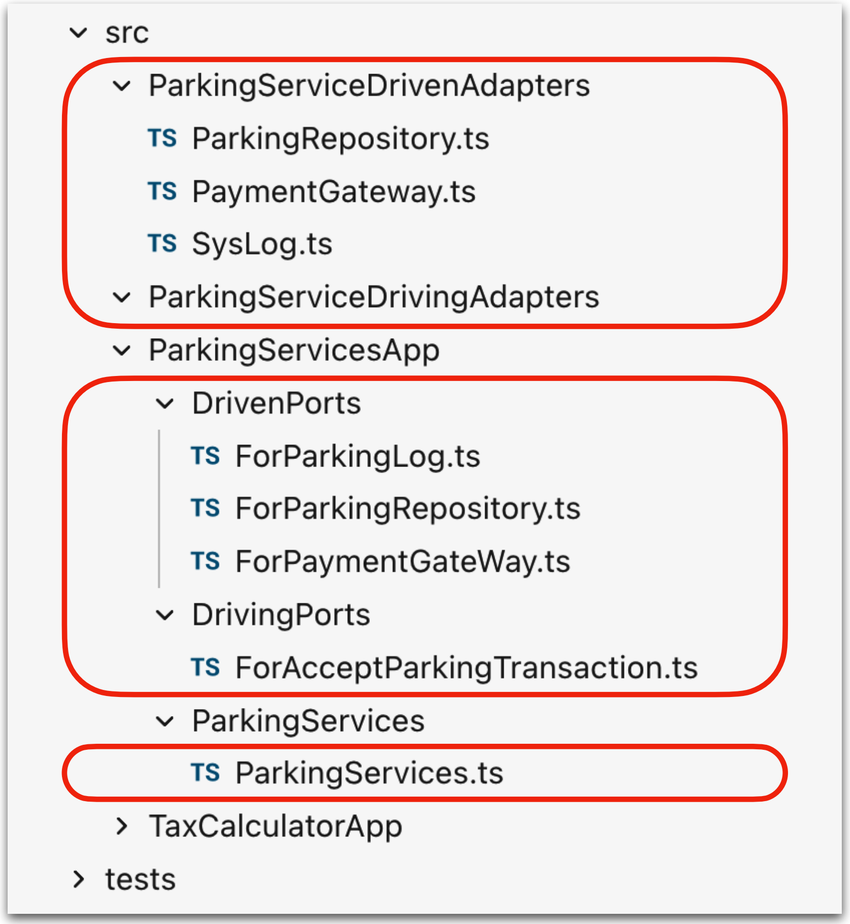
decouple
with ports

app not depends
on technology



Information Hiding

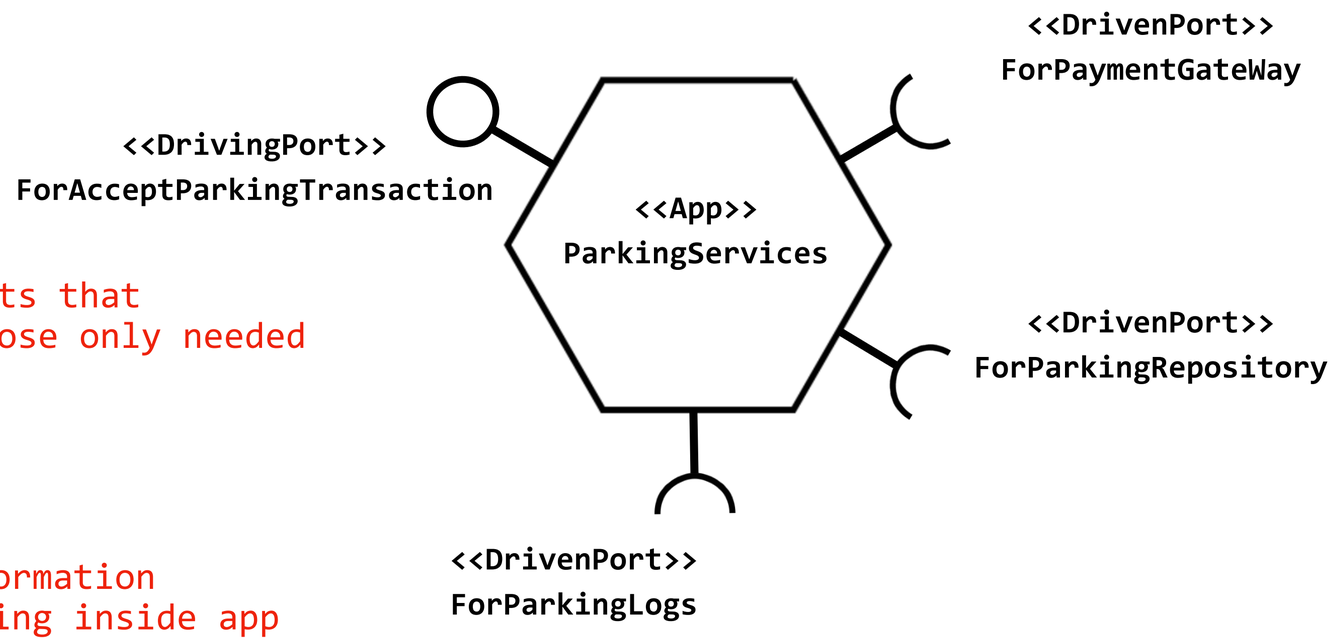
External client should not know too much about internal information & structure



adapters
implementation

ports that
expose only needed

information
hiding inside app



Nested Hexagonal

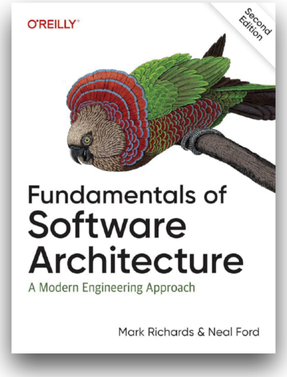
One more thing…

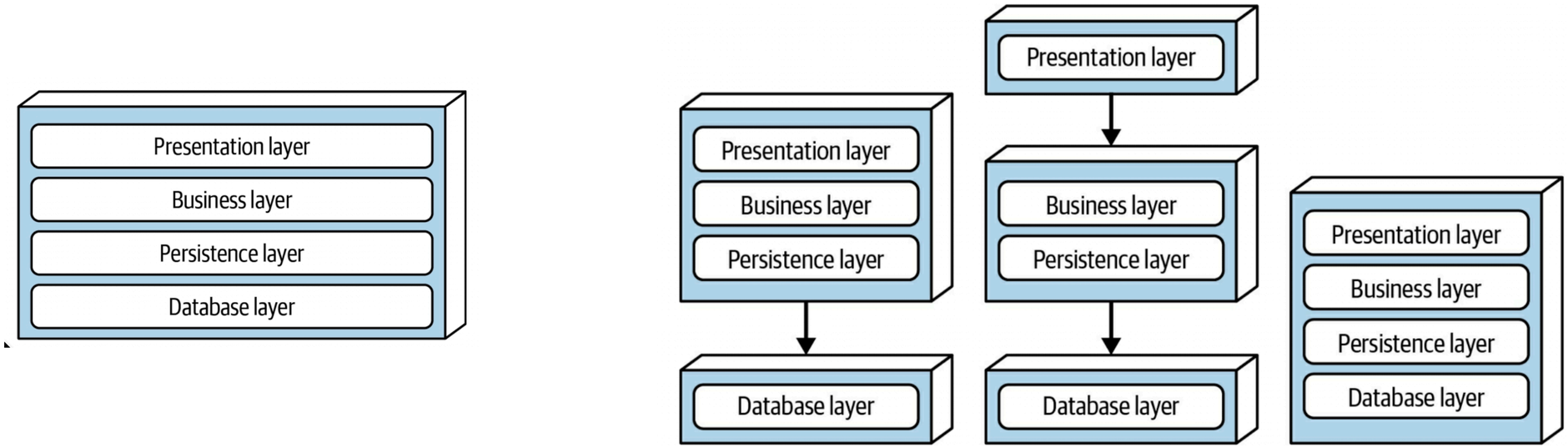
- You can have components within components if you like, assuming you meet the criteria.
- The Ports & Adapters pattern is aimed at protecting your team from external technology shifts. The pattern suggests declaring the system boundary just in front of each external technology.
- As such, we say that the Hexagonal / Ports & Adapters pattern does not nest.

From Big Ball of Mud to Modular Monolith with Ports & Adapters Pattern

Layer Architecture

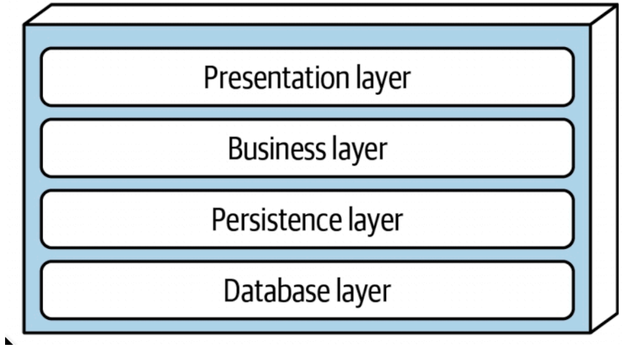
From The Fundamental of Software Architecture Book

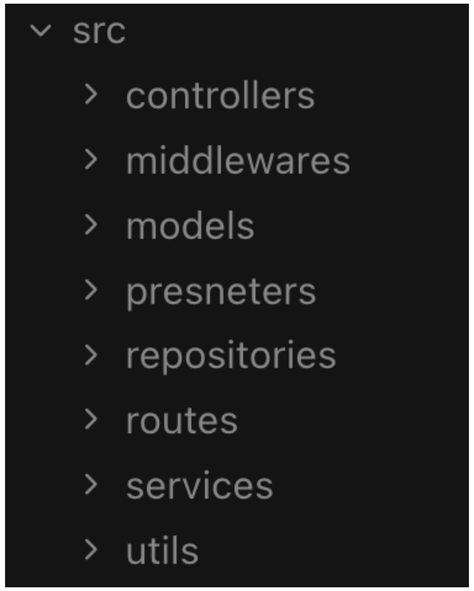




Layer Architecture

Without proper boundaries and discipline, layered architecture can degrade into a Big Ball of Mud.

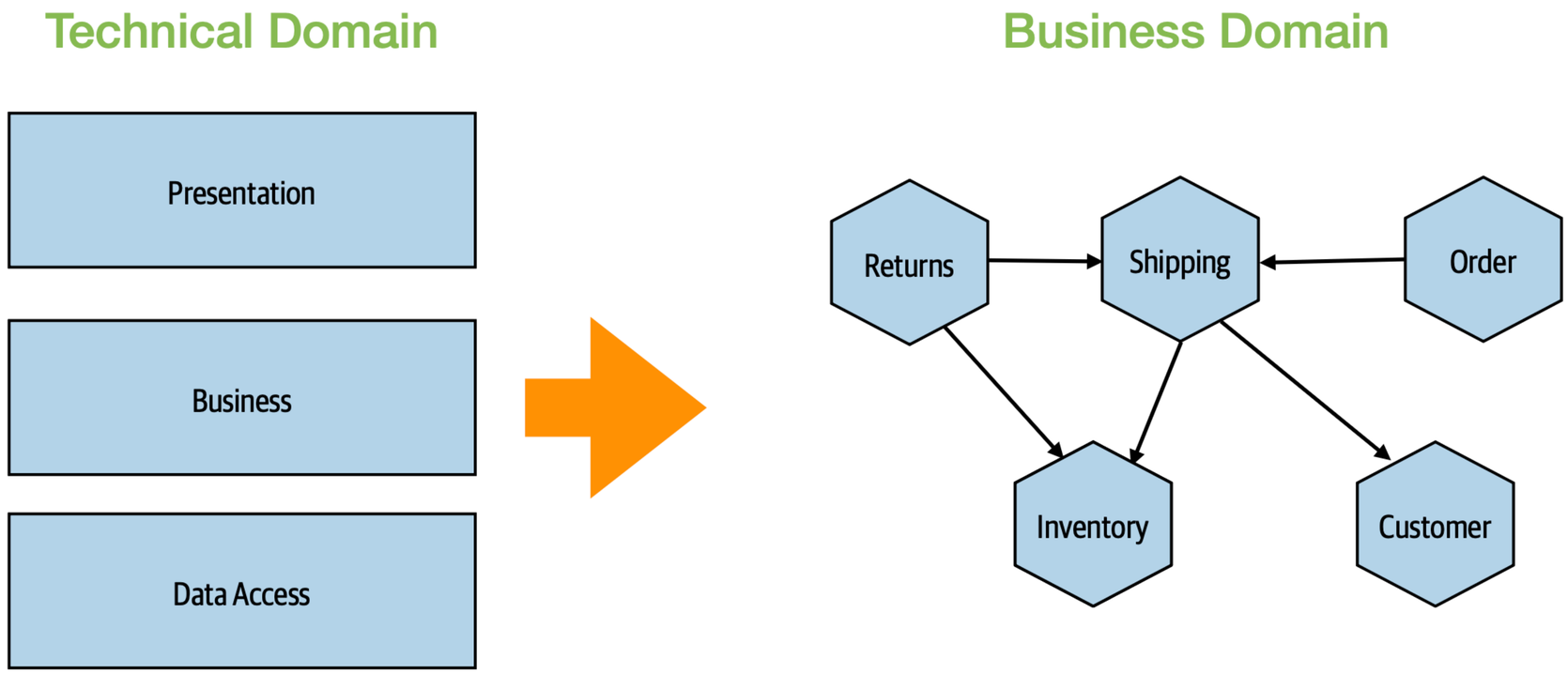





```
src/
  controllers/
    user.controller.ts
    order.controller.ts
    payment.controller.ts
    report.controller.ts
    admin.controller.ts
    shared.controller.ts

  services/
    user.service.ts
    order.service.ts
    payment.service.ts
    report.service.ts
    notification.service.ts
    auth.service.ts
    helper.service.ts
    common.service.ts
    integration.service.ts
    data.service.ts
    legacy.service.ts

  repositories/
    user.repository.ts
    order.repository.ts
    payment.repository.ts
    report.repository.ts
```

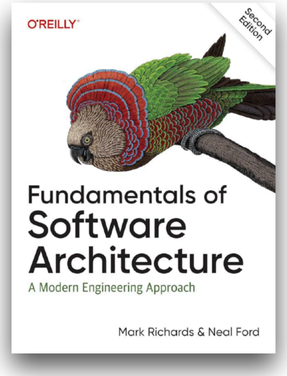
App Structure represents Business Structure

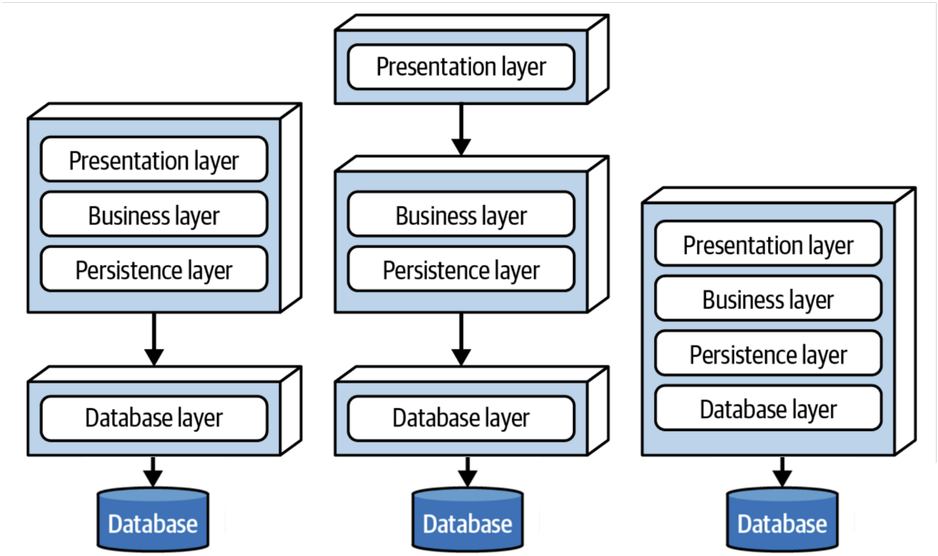
From technical perspective to business perspective



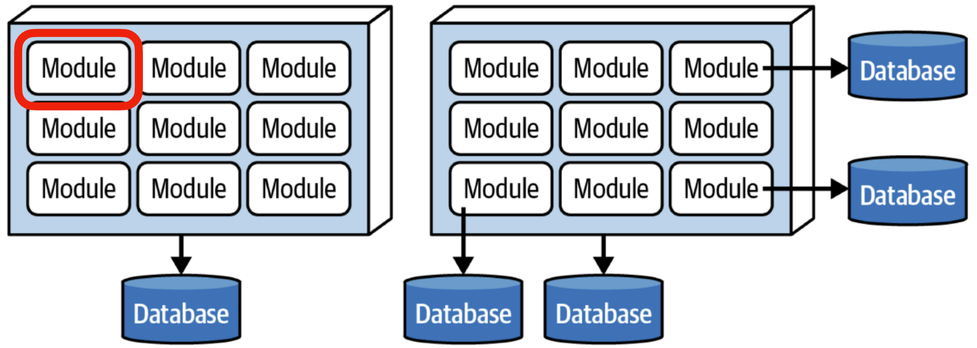
Layer vs. Modular Architecture

From The Fundamental of Software Architecture Book





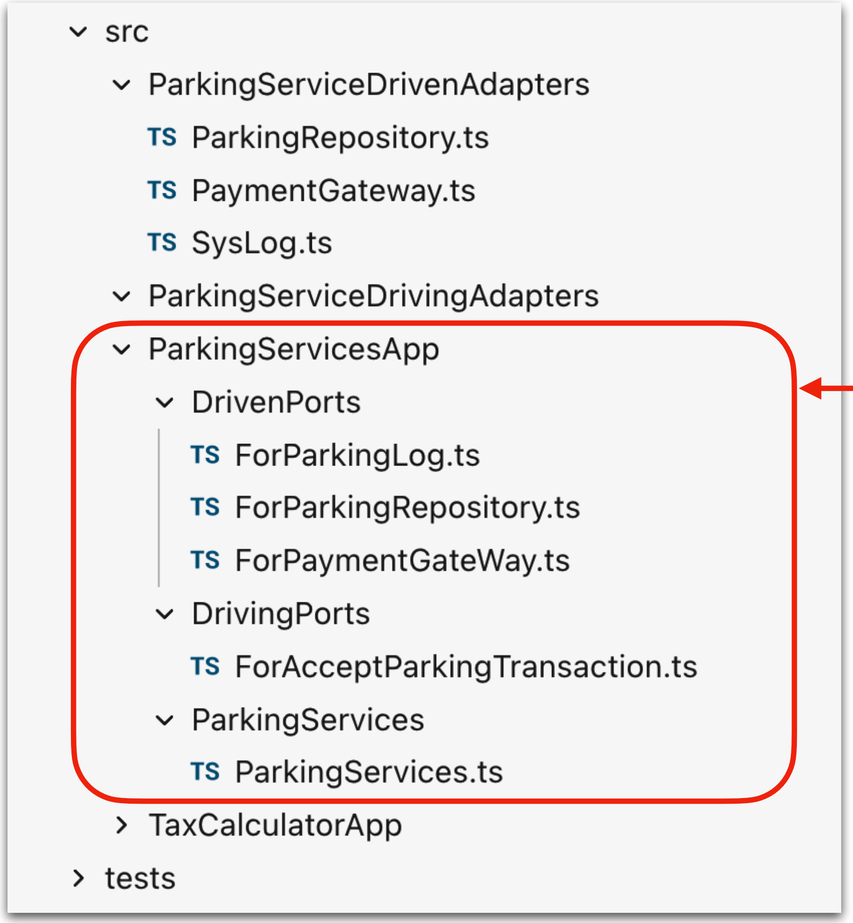
Layer Architecture

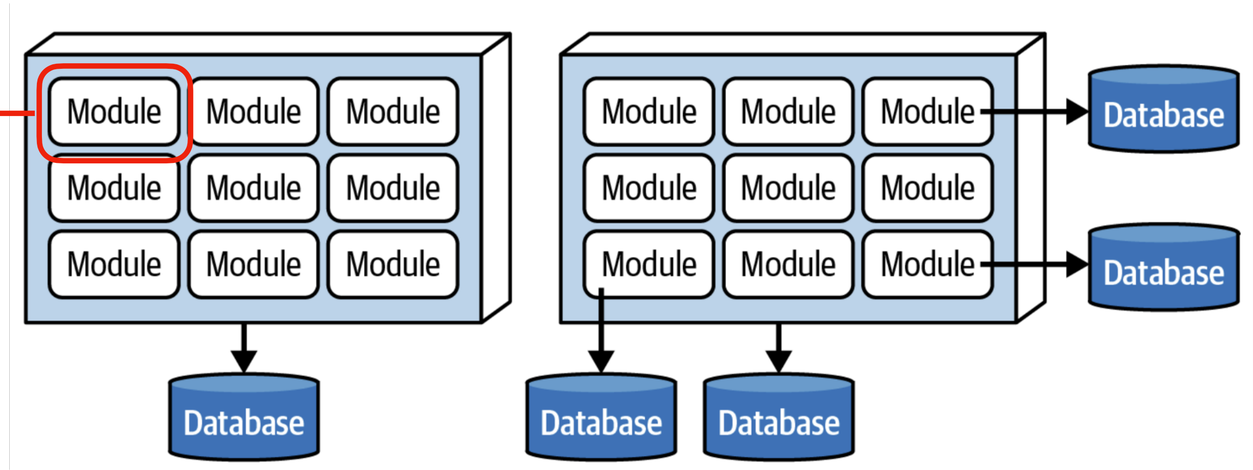


Modular Monolith Architecture

Hexagonal Architecture

In Modular Monolith





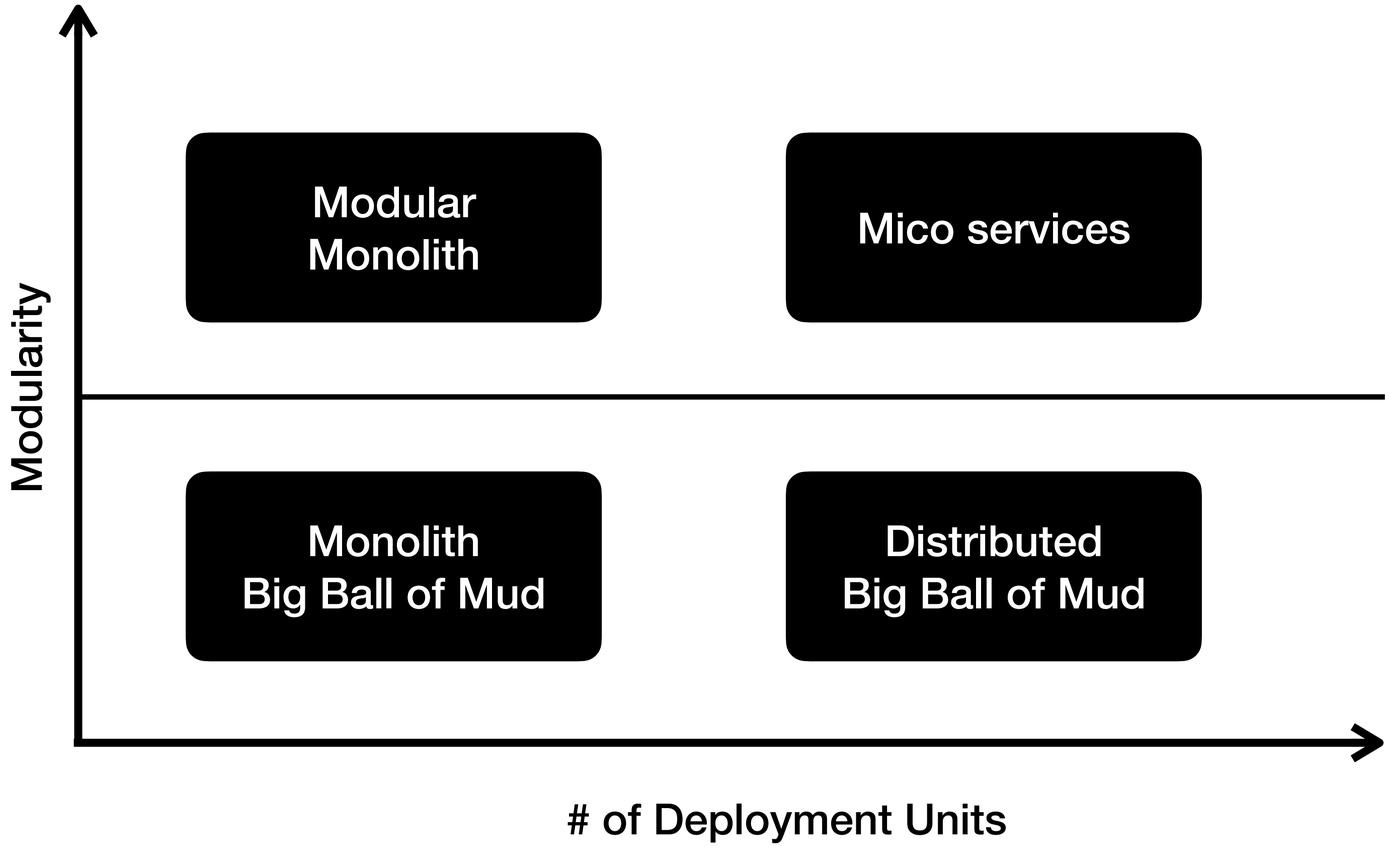
Exercise :-

Restructure app to become Modular Monolith with Ports & Adapters

- Set up Group of three.
- From “`04.Layered`” lab.
- Restructure app structure from layered to modular structure.
- Apply Ports & Adapters pattern as many module as you can.
- Reflection
  - What surprised you the most?
  - What patterns or themes stood out?
  - What topics sparked your curiosity or felt challenging?

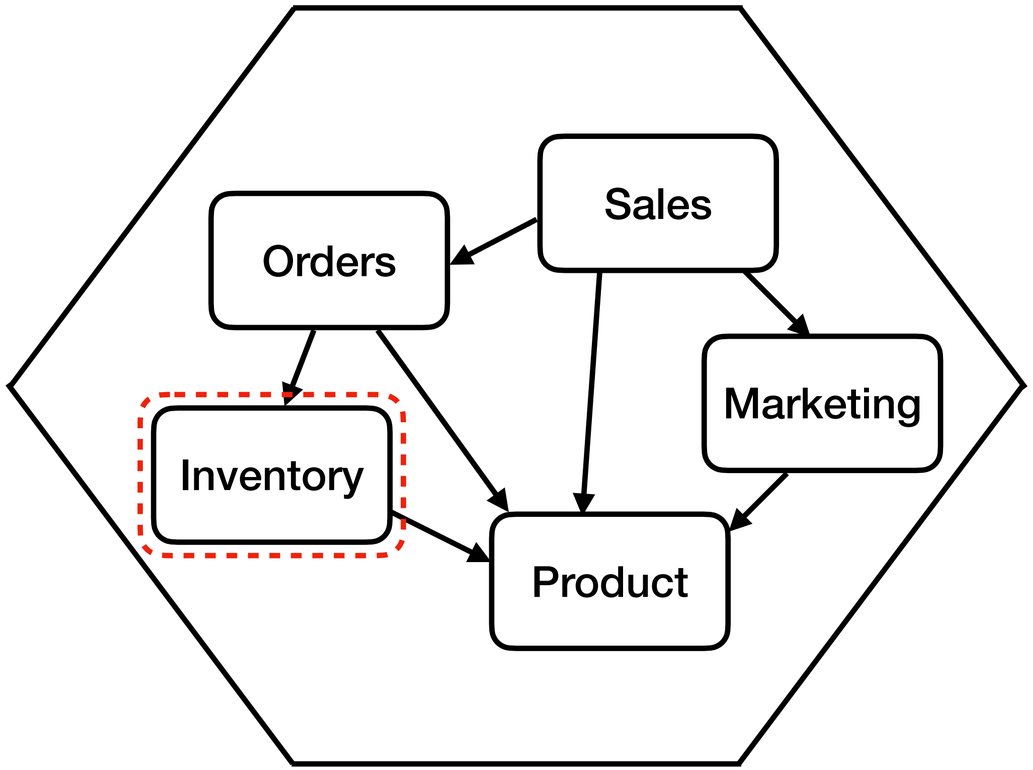
From Monolithic to Distributed Architecture with Ports & Adapters

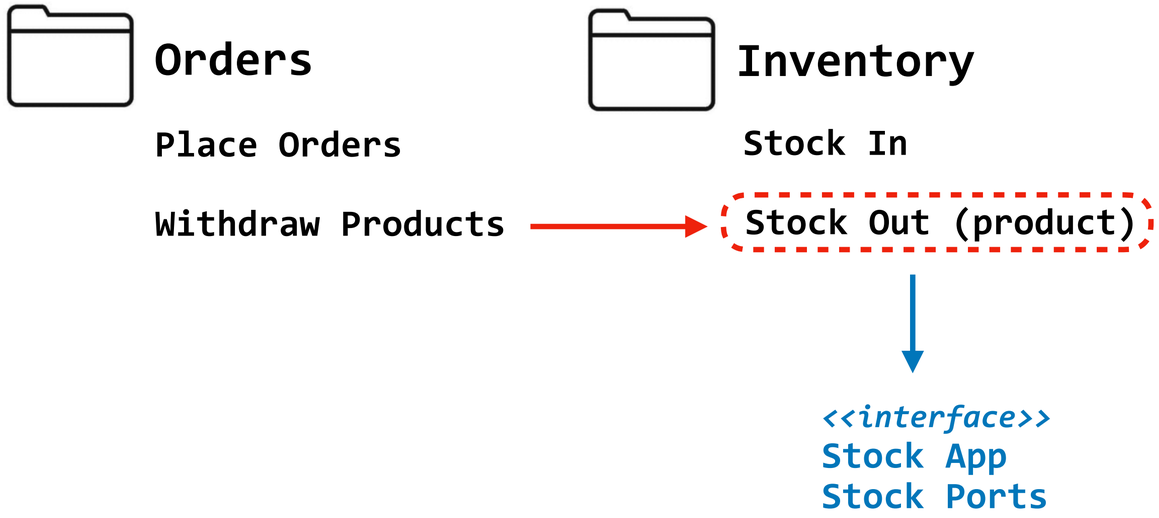
Monolith vs. Distributed Architecture



Breakdown Monolith

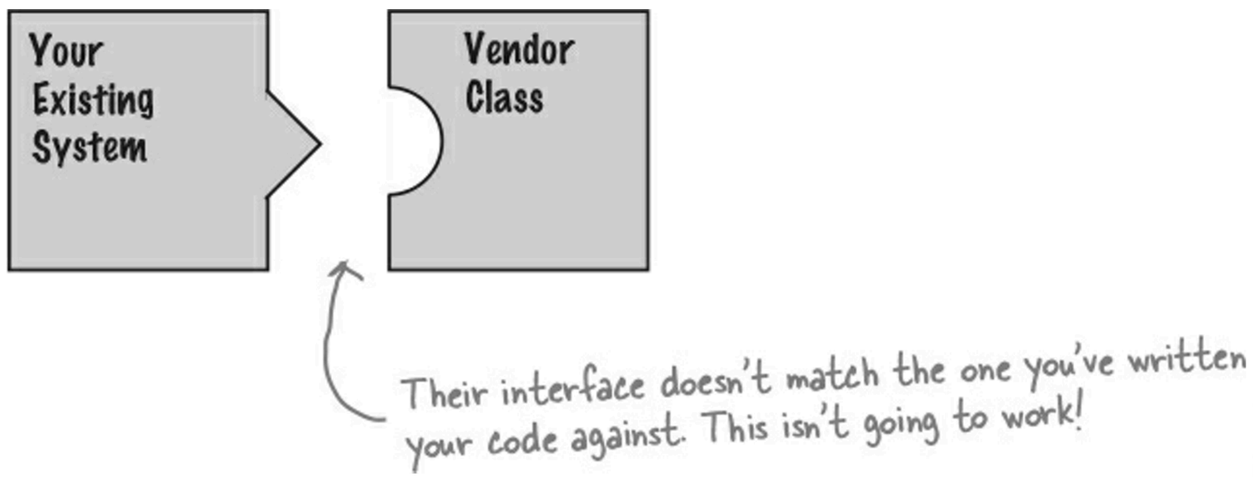
From Ports & Adapter Structure & Pattern

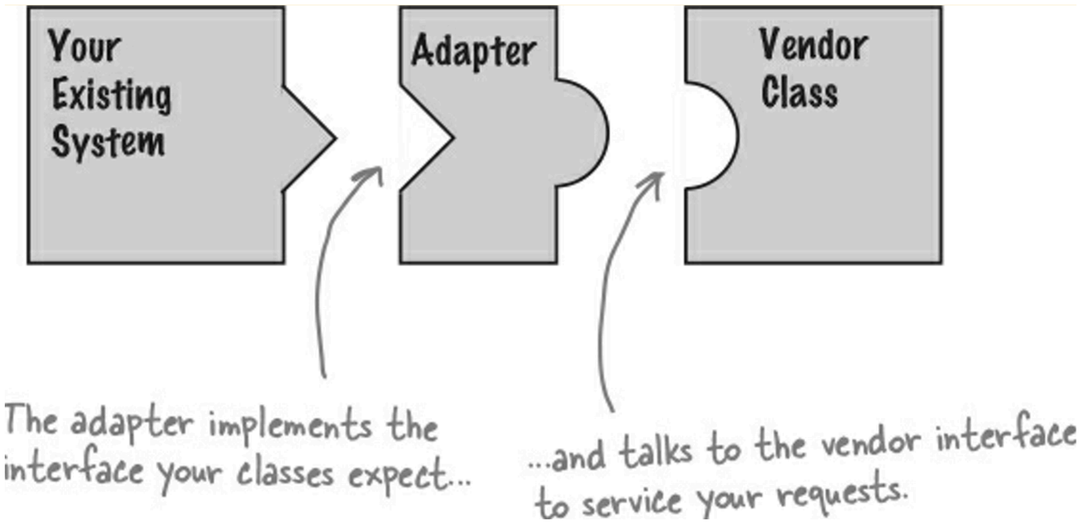




Using Adapter

Use it when you have an existing system structure but it incompatible with the vendor interfaces.

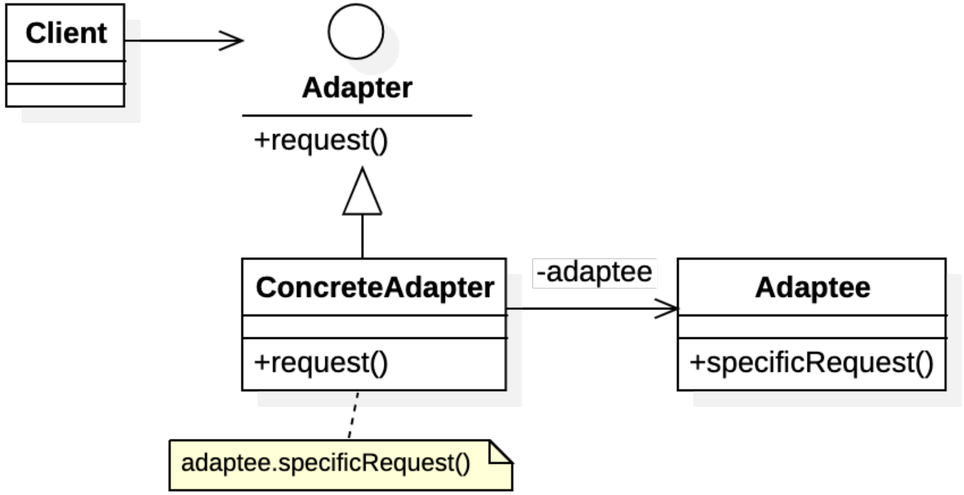


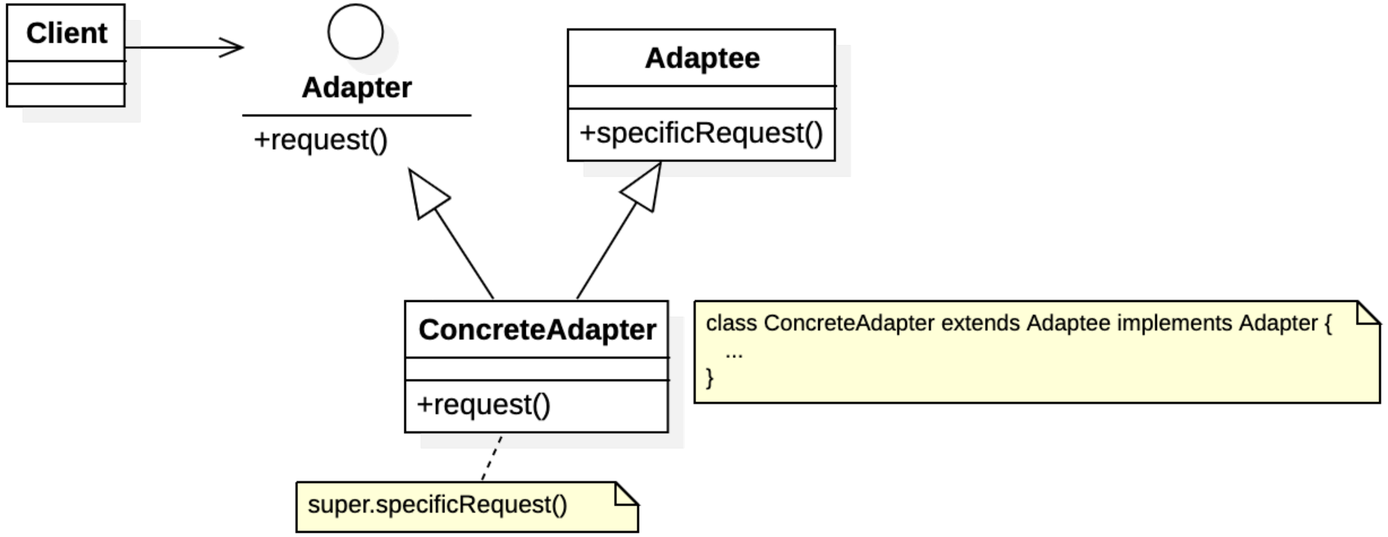


Doesn't alter the interface, but adds responsibility !!!

2 kinds of Adapters

UML class Diagram





Composition (Object Adapter)

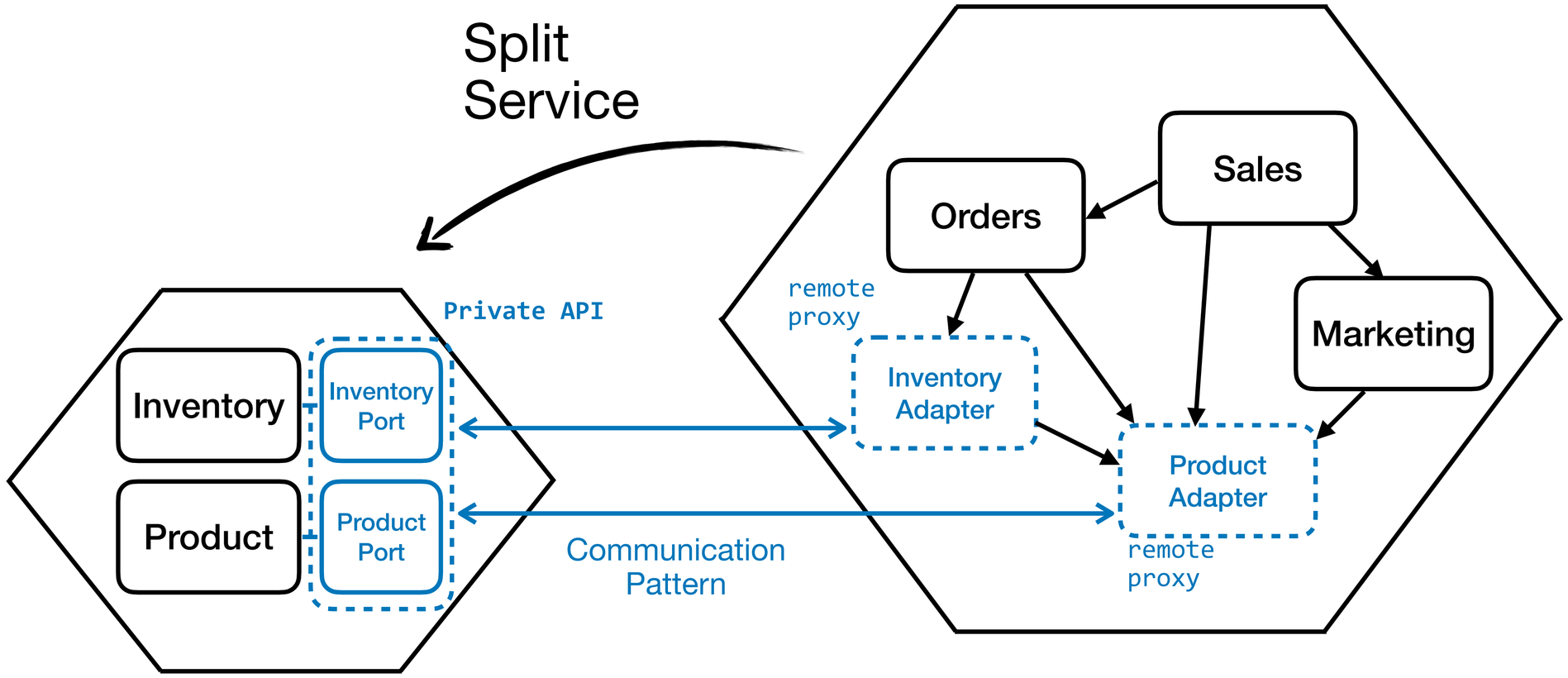
- Can work with many Adaptee sub-class.
- Re-implement the wrapped methods.
- More flexible.

Inheritance (Class Adapter - *If Adoptee class is not final or sealed*)

- Committed to one subclass (with many interfaces).
- Reuse implemented logic from super class.
- More reusable.

Breakdown Monolith

Communication over ports & adapters



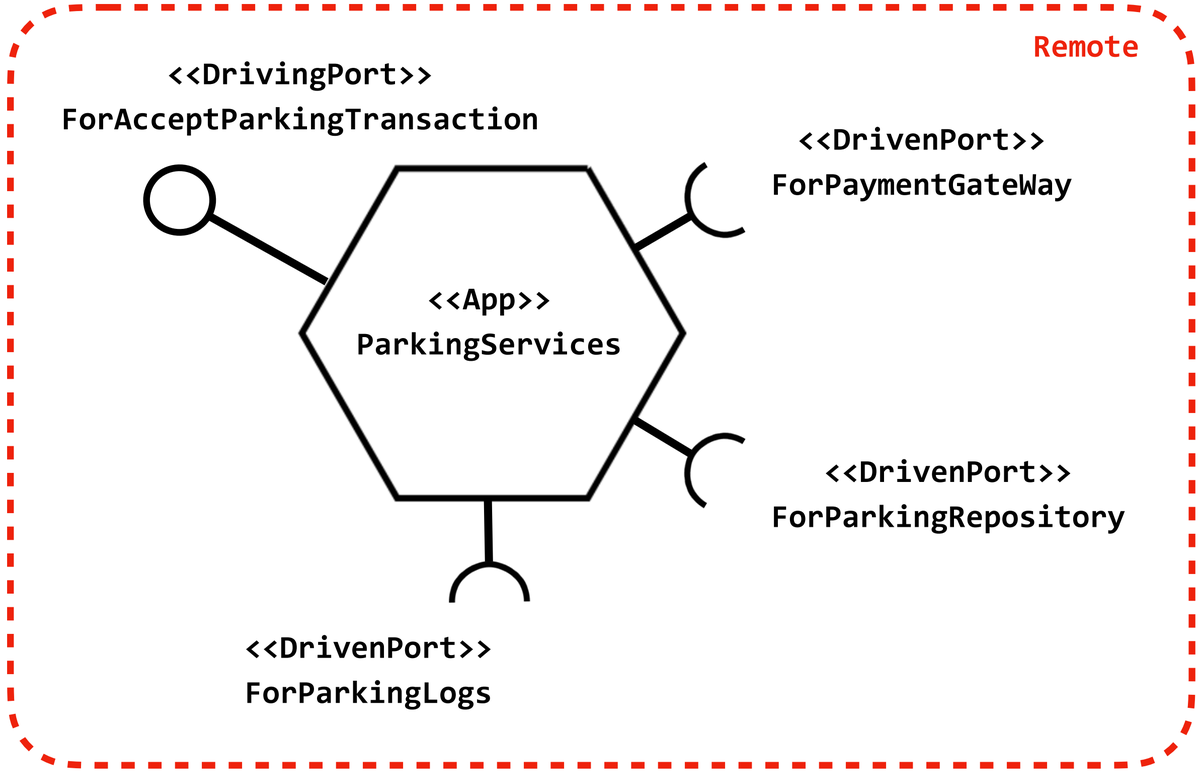
Exercise (1/2): Parking Lot

Splitting into another service.

```
type ParkingTransaction = Readonly<{
    plateNo: string
    parkingZone: string
    paymentPayload: PaymentPayload
}>
```

```
type PaymentPayload = Readonly<{
    transaction: string
    cardNo: string
    expireDate: Date
    ccv: string
}>
```

- From existing Parking Services Hexagonal…



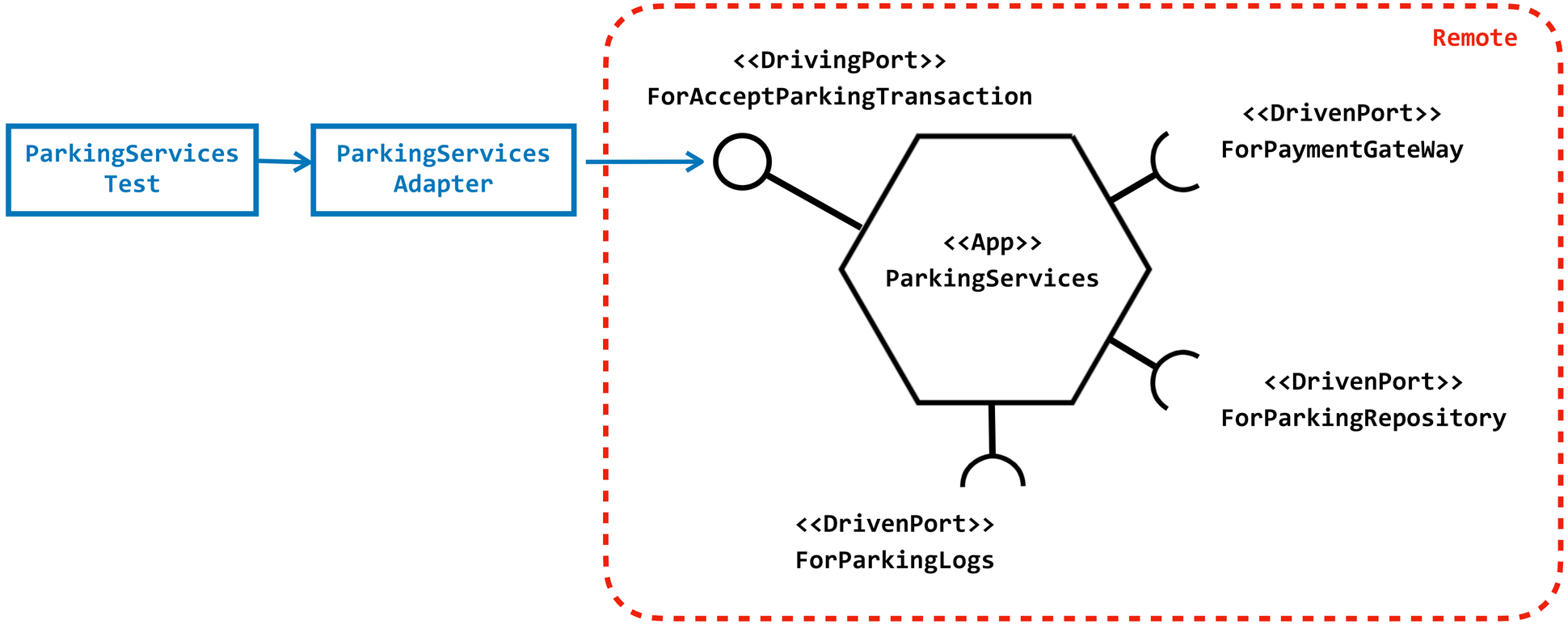
Exercise (2/2): Parking Lot

Splitting into another service.

```
type ParkingTransaction = Readonly<{
    plateNo: string
    parkingZone: string
    paymentPayload: PaymentPayload
}>
```

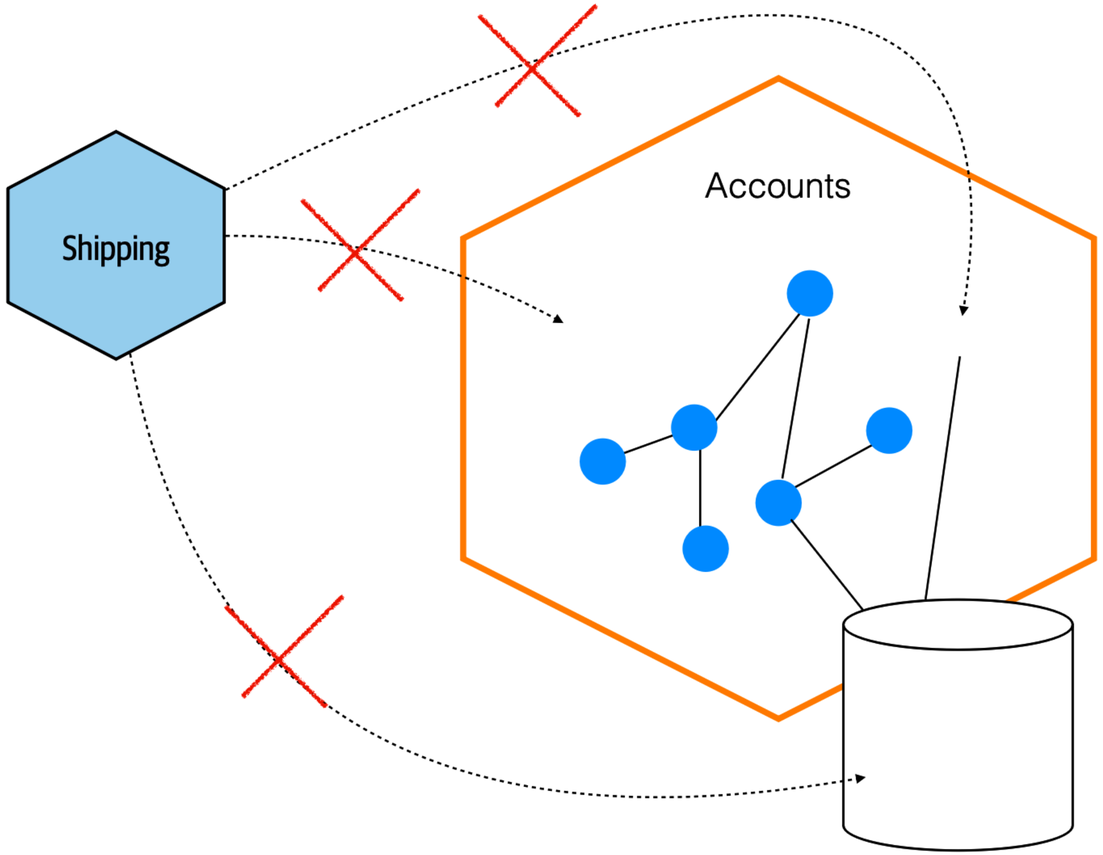
```
type PaymentPayload = Readonly<{
    transaction: string
    cardNo: string
    expireDate: Date
    ccv: string
}>
```

- Implement proxy remote call…



Information Hiding

External client should not know too much about internal information & structure

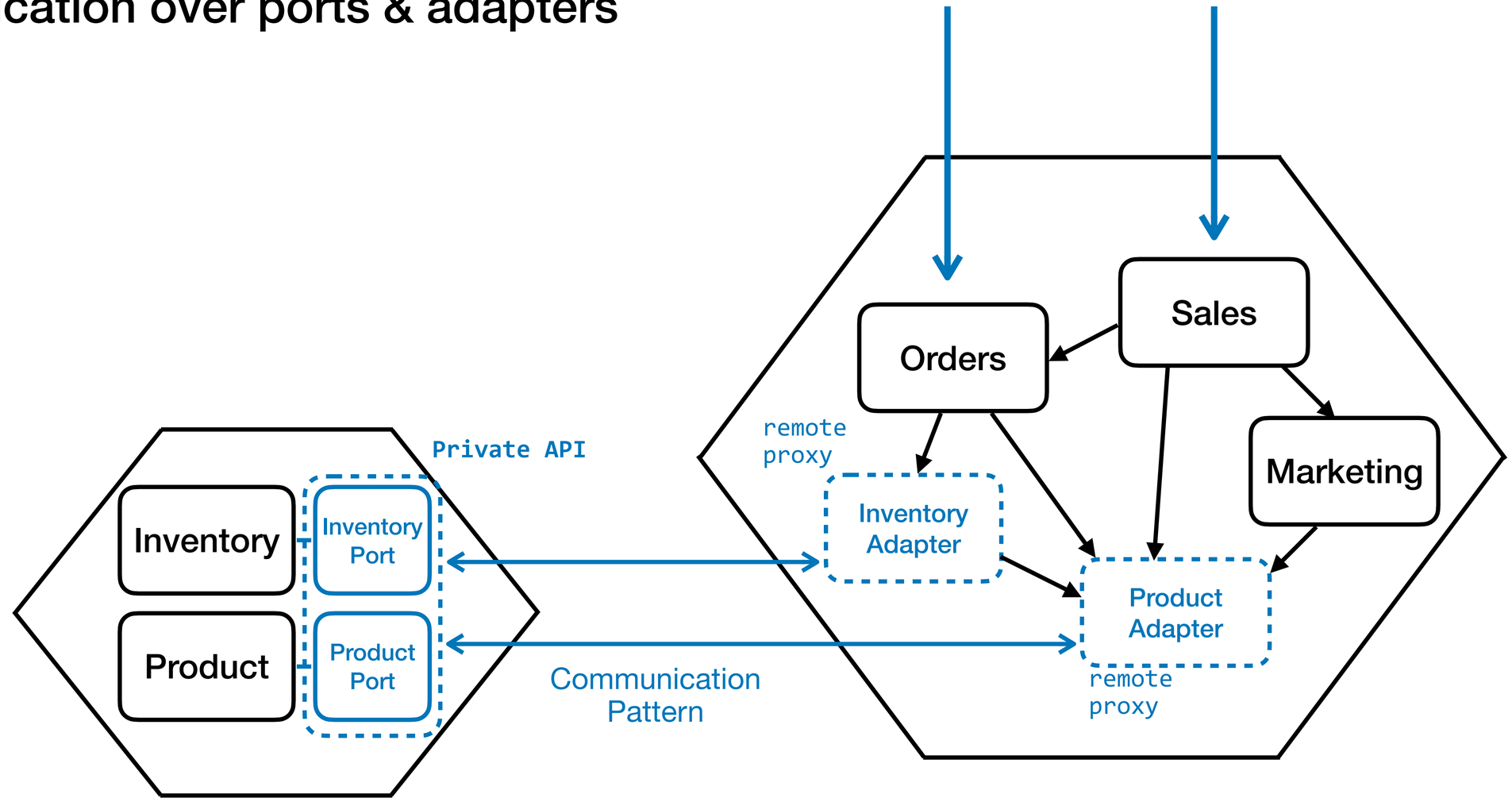


If an upstream consumer can reach into your internal implementation..

…then you can't change this implementation without breaking the consumer

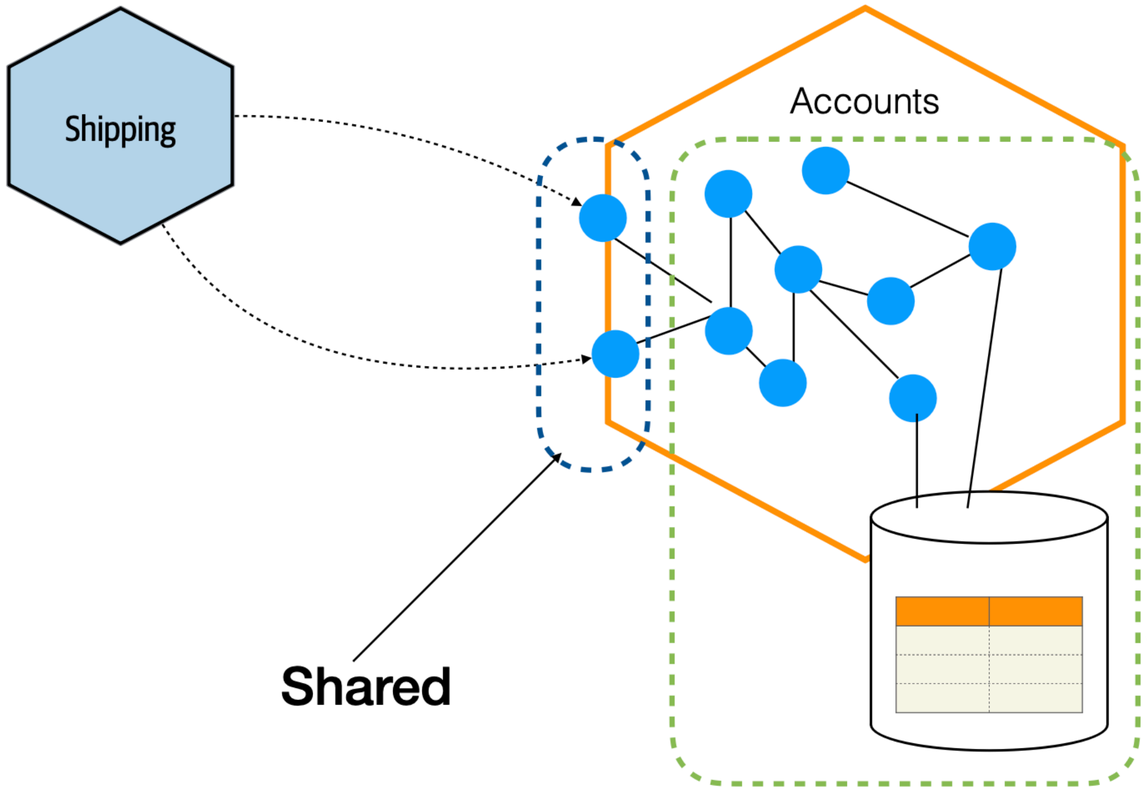
Breakdown Monolith

Communication over ports & adapters



Information Hiding

External client should not know too much about internal information & structure



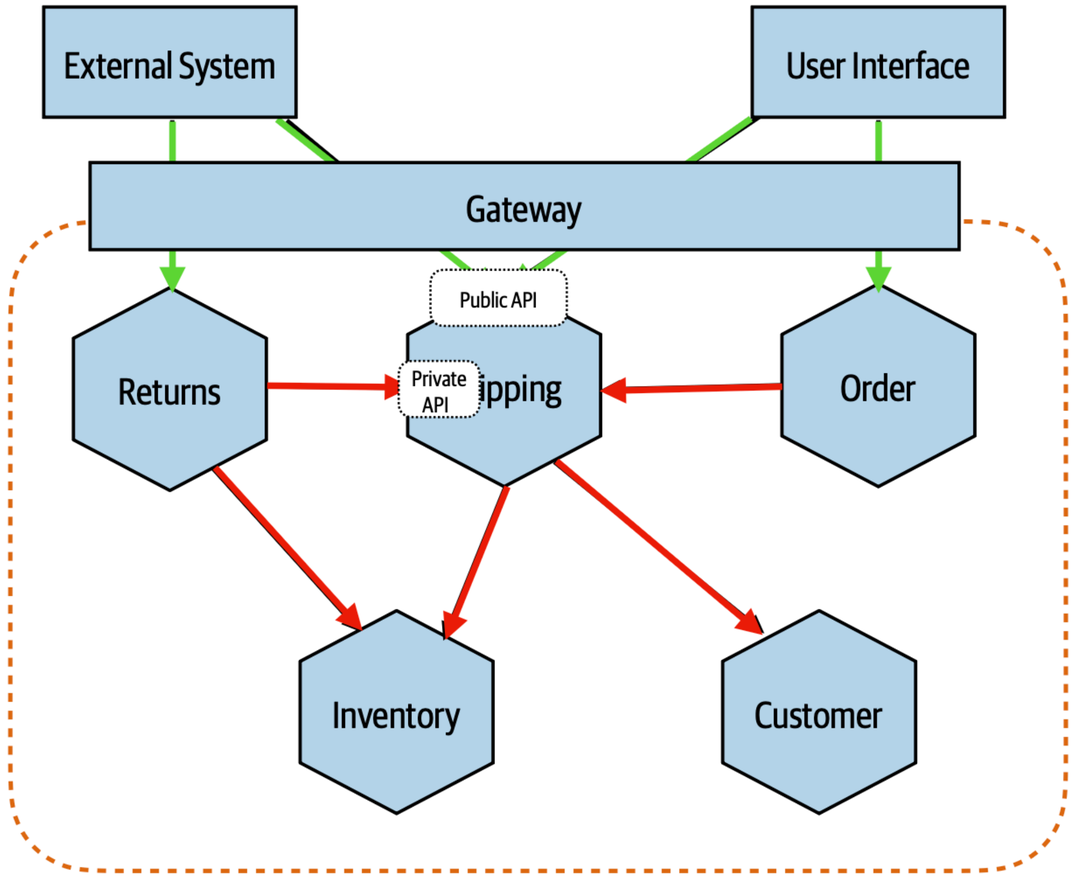
Hide your secrets!

Be explicit about what is shared, and what is hidden

Hidden things can change, shared things can't

The wider Ecosystem

Public API vs. Private API in Microservices Architecture

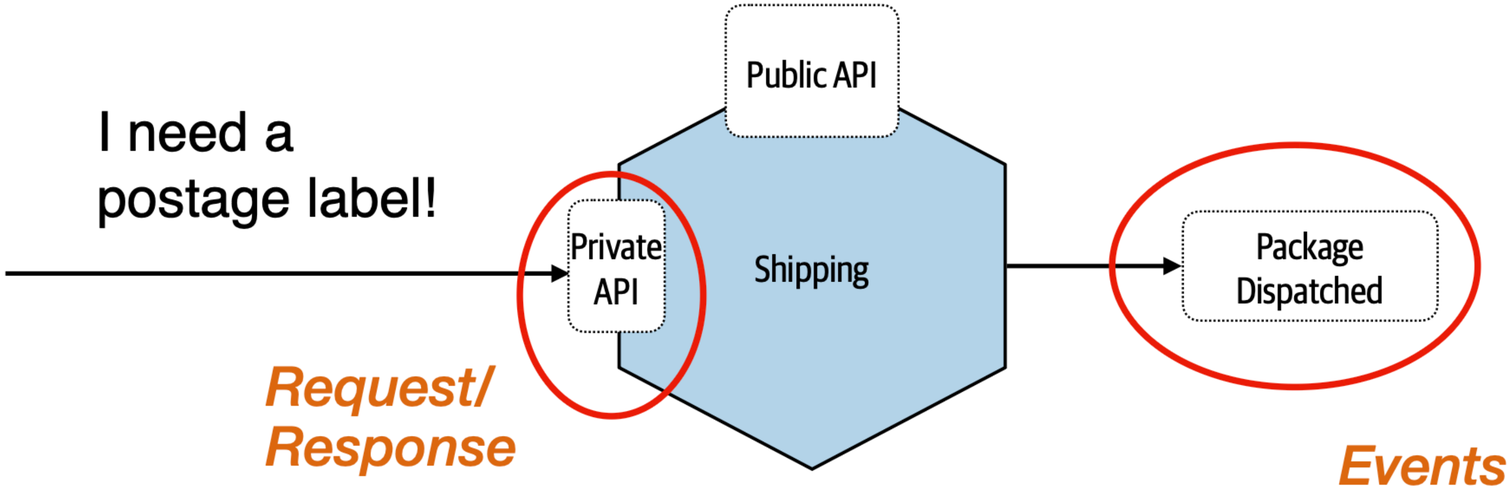


Comms within a perimeter - “east-west”

Comms in/out of the perimeter - “north-south”

Multiple Endpoints

Microservices point-of-view



A microservice can expose multiple different endpoints

More about the Software Architecture

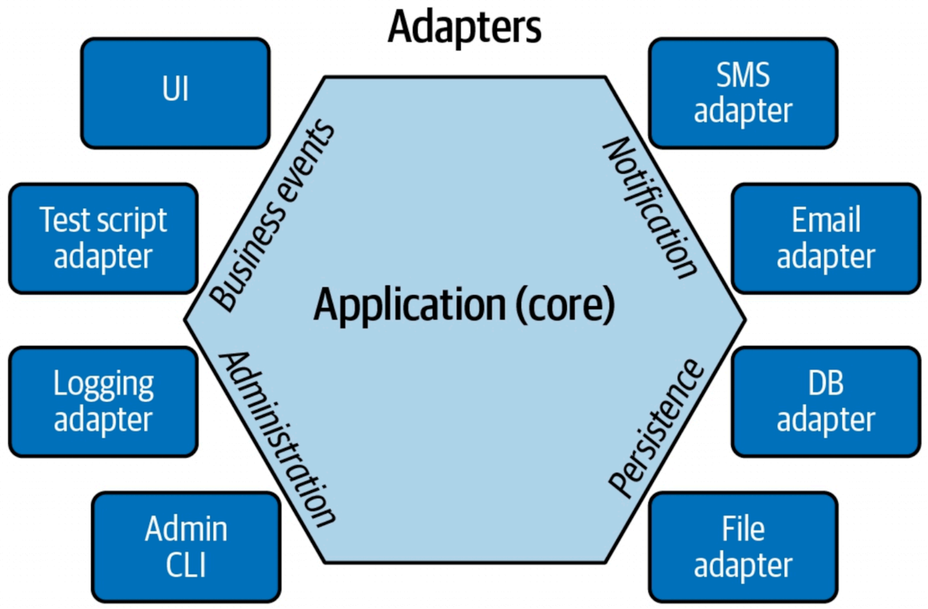
Ports & Adapters is only a small thing

However, it influences how we think about software architecture.

From The Inception film, 2010, Christopher Nolan : https://www.imdb.com/title/tt1375666/

Hexagonal Recap

The missed lead

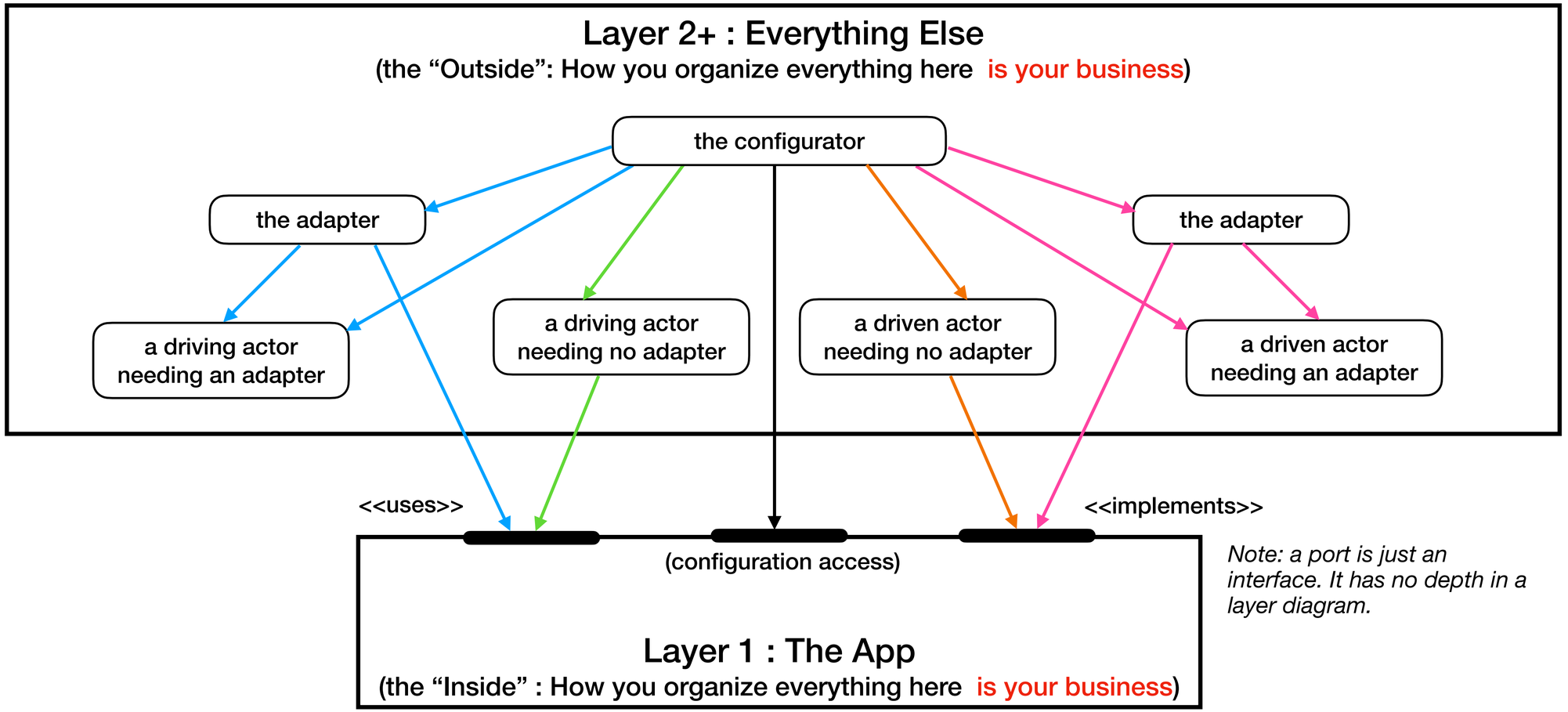


- This pattern's creator, Alistair Cockburn, initially drew it as a hexagon and called it the Hexagonal architecture pattern. He almost instantly regretted it. Since the name Ports and Adapters is more descriptive, but it was too late.
- Too many architects thought "Hexagonal" sounded cool, so the name stuck.
- The most confusing is Hexagonal when used to describe Microservices Architecture is it treats the database as just another adapter that leads to the data schema was an entirely separate piece of machinery. Until Eric Evan correct this mistake in his book, Domain-Driven Design.
- One way to solved this problem in the Microservices Architecture Design is use Service Mesh.
- However, as long as you are not miss understanding about "separate of domain and operational concern". IT would be fine to use it.

Reference :- Fundamental of Software Architecture (2nd Edition) - Chapter 20. Architectural Patterns

Inside & Outside

What Hexagonal Architecture care about.



Note: a port is just an interface. It has no depth in a layer diagram.

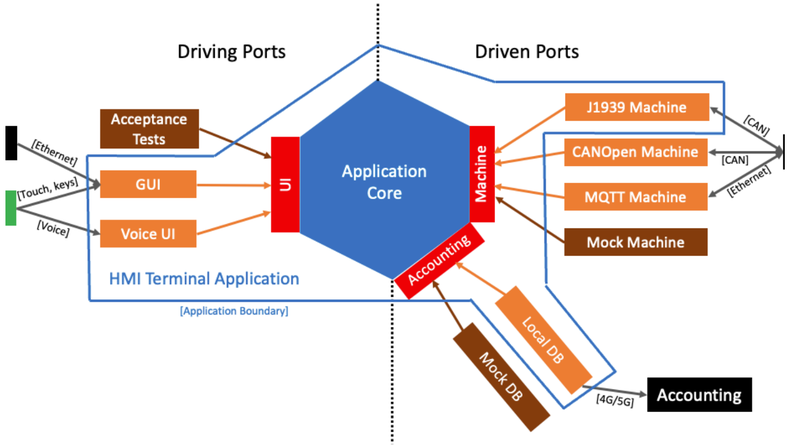
Hexagonal Architecture

A brief history

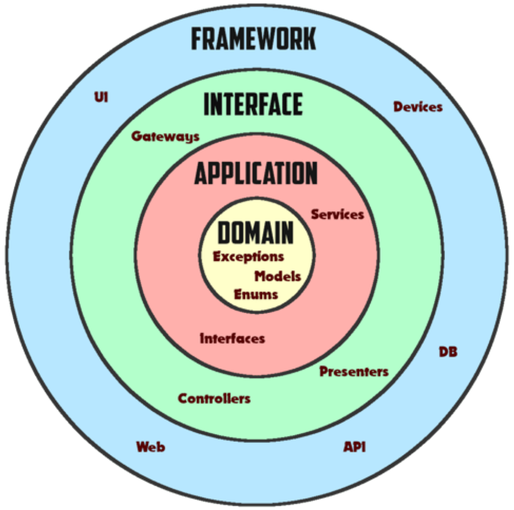
- Alistair Cockburn name ***Hexagonal Architecture*** years after he understood what the sides of the hexagonal stood for.
  - 1994: On a fixed-price, fixed-time project involving an object-relational mapper, the infrastructure designers found they had to change their design to the SQL database to improve performance. They instead shut down the project for several weeks and frantically rewrote their mapper.
  - In 2000, Alistair Cockburn helped a friend struggling with multiple input sources and notification variants by applying what became the Ports and Adapters pattern.
  - 2005: Alistair renamed the pattern to the more accurate “Ports & Adapters” and wrote it up with code samples.
  - 2015: The notions of driving and driven adapters were added, along with the naming convention for interfaces as “for_doing_something”.
  - 2022: The pattern as a special case of “Component + Strategy” was formulated, along with inclusion of the required interfaces concept.
  - 2023: The more complete and accurate Configurable Receiver pattern replaced the earlier, slight incorrect pattern “Configurable Dependency”.
  - 2024: Juan and Alistair complete their collaboration to bring you the preview edition of this book.

Hexagonal vs. Onion vs. Clean

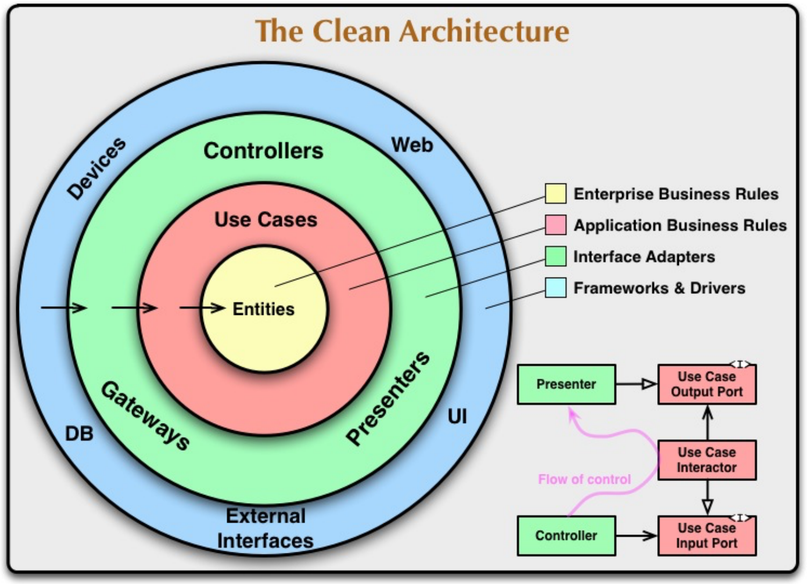
What related in between those architecture.



Hexagonal



Onion

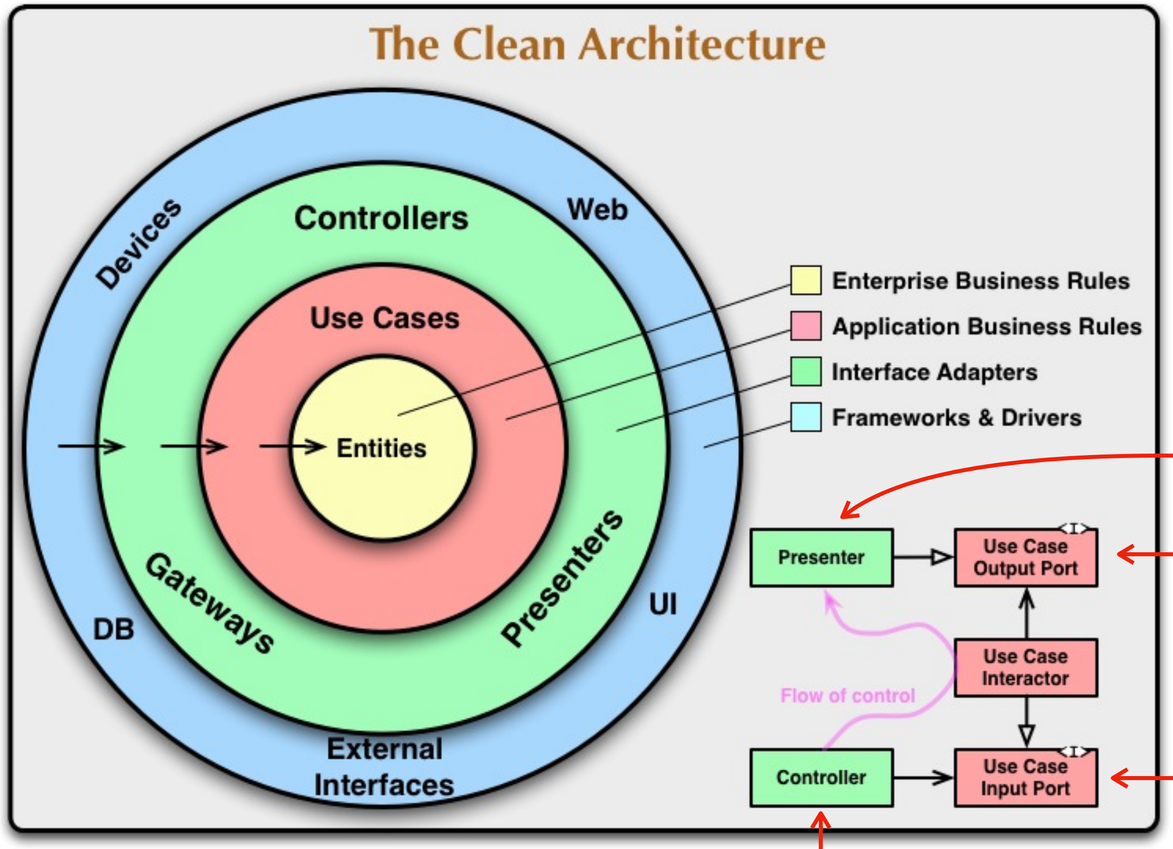


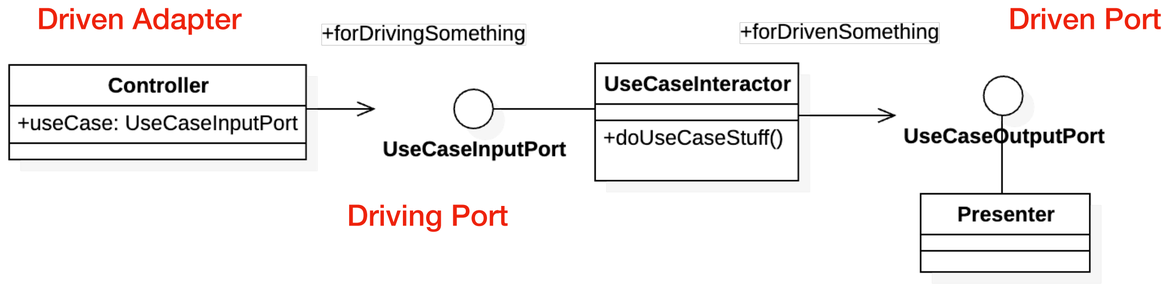
Clean

References :-
Ports & Adapter Blog : https://embeddeduse.com/2023/08/24/ports-and-adapters-architecture-the-pattern/
Onion Architecture Blog : https://softwareengineering.stackexchange.com/questions/450423/how-to-structure-libraries-solution-for-onion-architecture-with-model-view-prese
Clean Architecture Blog : https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

Clean Architecture

Let’s zoom in a little bit.



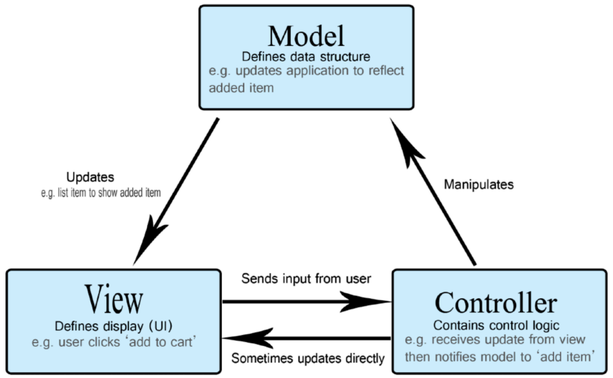


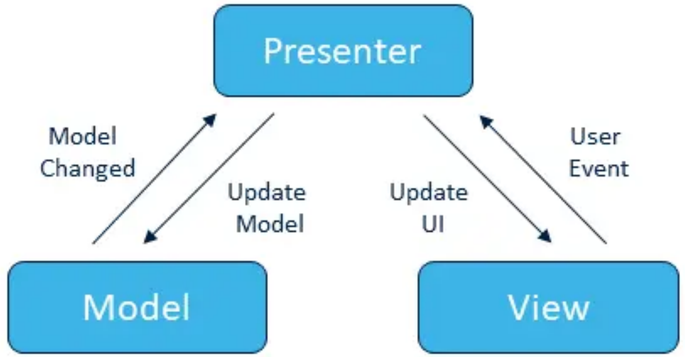
Note: This class does not cover the advanced concepts of the Clean Architecture.

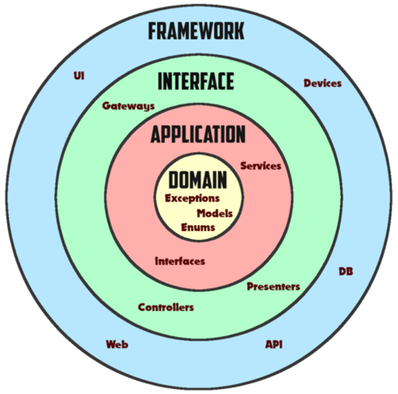
This page show only how related between Hexagonal and Clean Architecture.

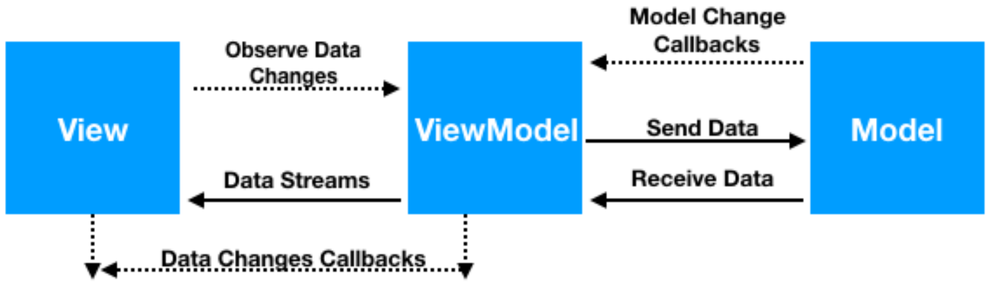
Another Application Architecture

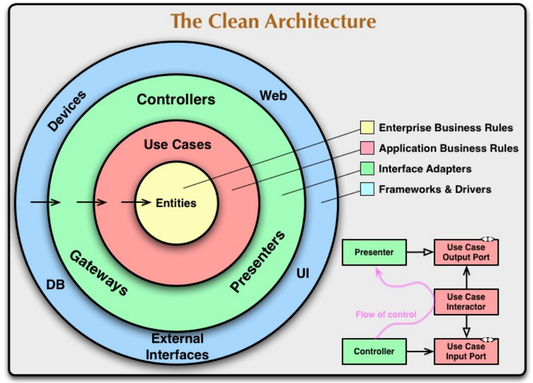
Separation of Concerns









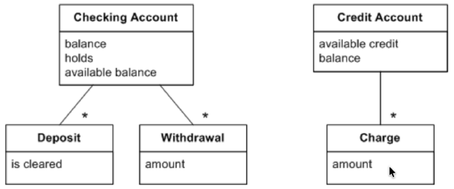


From Hexagonal point-of-view. As long as the business model and technology are separated. It will be OK. Another additional layers is up to your choice.

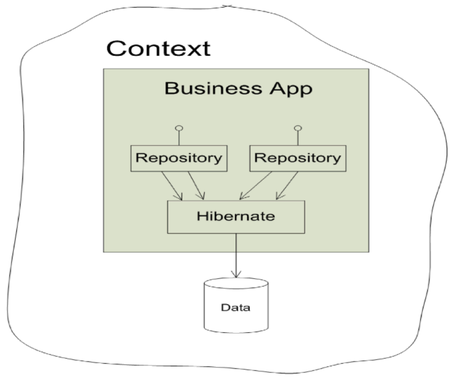
DDD Related

Working with Legacy with Ports & Adapters Apply

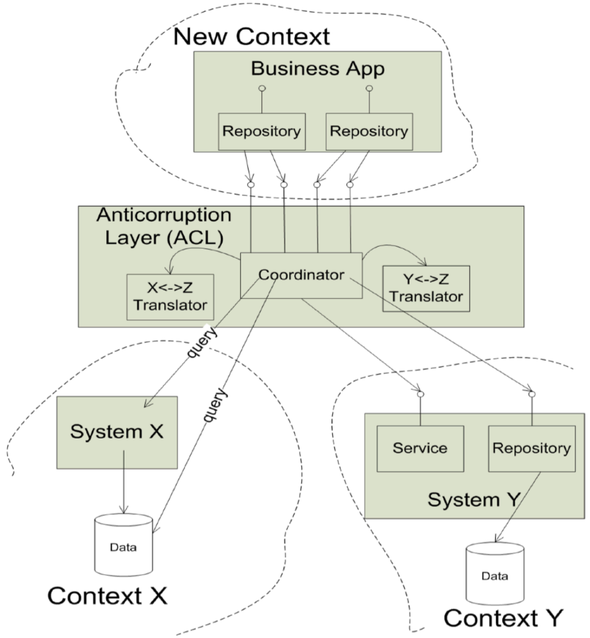
Existing model in Context A



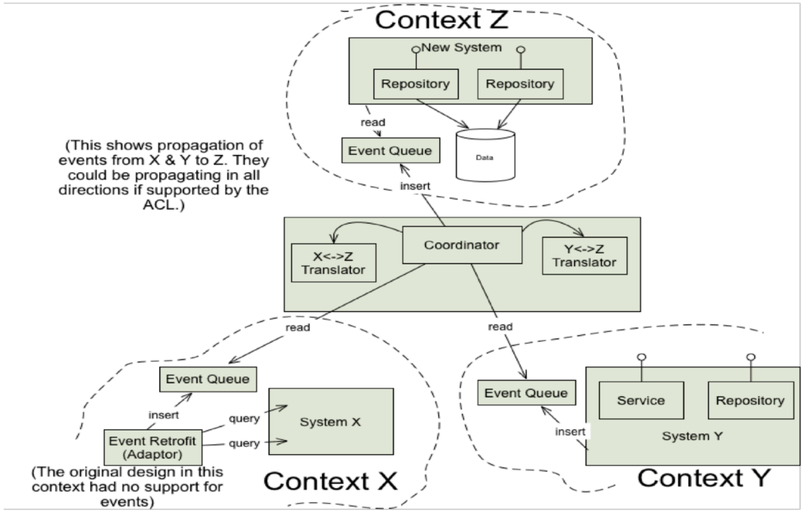
Classic Repository implementation



ACL-Backed Repository implementation



ACL translating Domain Events

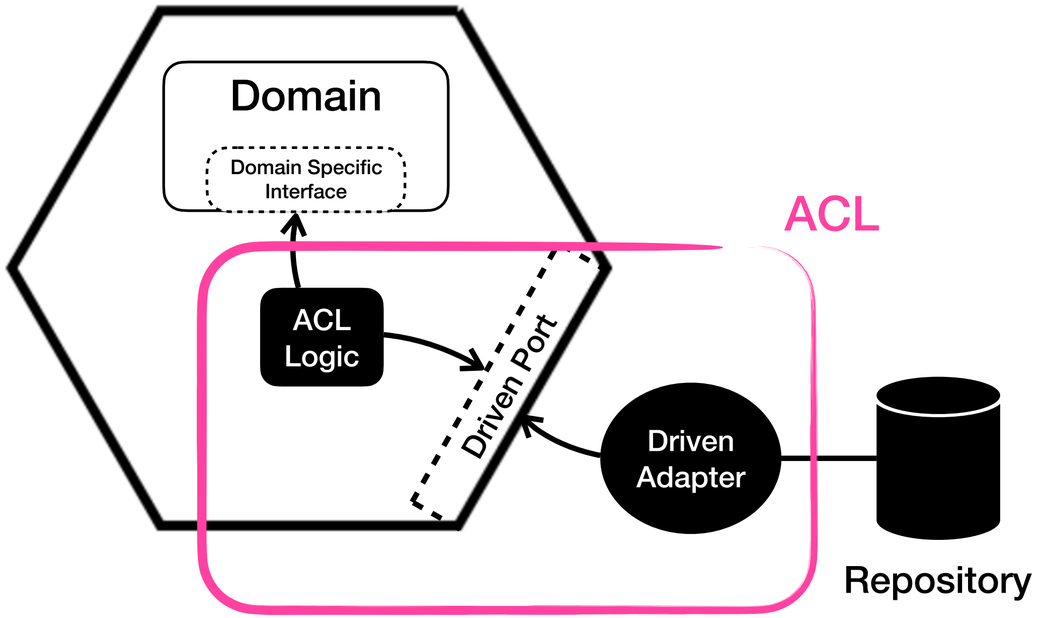


Reference :- Getting started with DDD then surrounded by legacy systems
https://www.domainlanguage.com/wp-content/uploads/2016/04/GettingStartedWithDDDWhenSurroundedByLegacySystemsV1.pdf

DDD Related

How does the Ports & Adapter related to Domain-Driven Design or DDD?

- First, they’re exist independently and evolve separately. They’re not depends on each other. However, they work well together.
- You can use apply bounded context in DDD with Ports and Adapters. But it’s not always necessary.
- You can make each bounded context its own component, define the ports, write the tests, and put the ACLs outside all of them, translating between them.
- You can put ports and boundaries around every bounded context, the Ports & Adapters pattern is really aimed at protecting your team from external technology shifts.
- The pattern suggests placing the system boundary just in front of each external technology, keeping the adapters simpler.



Q&A

References

Books and Blog

- Books & Documents:
  - Hexagonal Architecture Explained, Alistair Cockburn
    https://www.amazon.com/dp/B0F5F7YRWW?ref_=cm_sw_r_ffobk_cp_ud_dp_S34CK495KK4ATCKCTND2
  - Fundamentals of Software Architecture, 2nd Edition
    https://www.amazon.com/dp/B0F1BWQGYZ?ref_=cm_sw_r_ffobk_cp_ud_dp_9BC50PEBMJJB0964EVAG
  - Getting Started with DDD when surrounded by Legacy Systems by Eric Evan
    https://cqrs.wordpress.com/wp-content/uploads/2010/11/cqrs_documents.pdf
  - CQRS Document by Greg Young
    https://cqrs.wordpress.com/wp-content/uploads/2010/11/cqrs_documents.pdf
- Related Blog:
  - Ports & Adapter Blog
    https://embeddeduse.com/2023/08/24/ports-and-adapters-architecture-the-pattern/
  - Onion Architecture Blog
    https://softwareengineering.stackexchange.com/questions/450423/how-to-structure-libraries-solution-for-onion-architecture-with-model-view-prese
  - Clean Architecture Blog
    https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html